SESSION 4

05.10.2024

Agenda

Module-2 Data Analysis and Visualization with Python

Data Loading and Cleaning:Reading and Writing Data with Pandas,

Data Manipulation with Pandas,

Exploratory Data Analysis (EDA):Descriptive Statistics,

Data Visualization with Matplotlib and Seaborn,

Correlation and Covariance, Advanced Data Visualization-Interactive Plots with Plotly.

Lab 4 Load a dataset into a Pandas DataFrame and clean the data (handle missing values, drop unnecessary columns) and Perform group by operations and calculate summary statistics.

Lab 5 Create a variety of plots using Matplotlib,Use Seaborn to create advanced visualizations (box plot, heatmap) and customize them (titles, labels, colors).

Connecting MySQL with Python

To create a connection between the MySQL database and Python, the **connect()** method of **mysql.connector** module is used. We pass the database details like HostName, username, and the password in the method call, and then the method returns the connection object.

The following steps are required to connect SQL with Python:

Step 1: Download and Install the free MySQL database.

Step 2: After installing the MySQL database, open your Command prompt.

Step 3: Navigate your Command prompt to the location of PIP.

Step 4: Now run the commands given below to download and install “MySQL Connector”. Here, mysql.connector statement will help you to communicate with the MySQL database.

Download and install “MySQL Connector”

pip install mysql-connector-python

Step 5: Test MySQL Connector

To check if the installation was successful, or if you already installed “MySQL Connector”, go to your IDE and run the given below code :

import mysql.connector

If the above code gets executed with no errors, “MySQL Connector” is ready to be used.

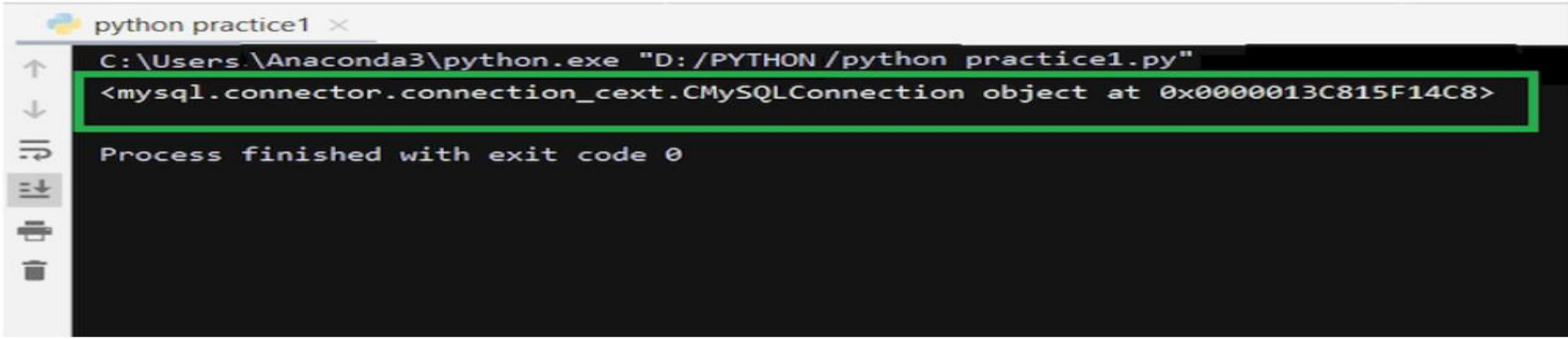
Step 6: Create Connection

Now to connect SQL with Python, run the code given below in your IDE.

 # Importing module

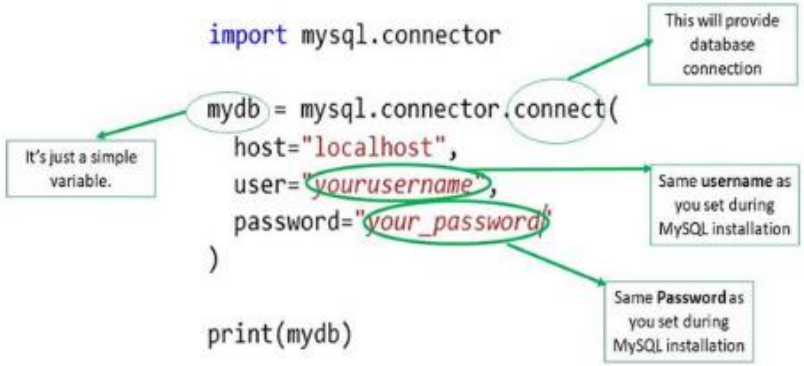
import mysql.connector

Output:



```
# Creating connection object
mydb = mysql.connector.connect(
    host = "localhost",
    user = "yourusername",
    password = "your_password"
)
# Printing the connection object
print(mydb)
```

Here, in the above code:



Database Access in Python

Database access in Python is used to interact with databases, allowing applications to store, retrieve, update, and manage data consistently.

Various relational database management systems (RDBMS) are supported for these tasks, each requiring specific Python packages for connectivity,

- GadFly
- MySQL
- PostgreSQL
- Microsoft SQL Server
- Informix
- Oracle
- Sybase
- SQLite
- and many more...

Data input and generated during execution of a program is stored in RAM.

If it is to be stored persistently, it needs to be stored in database tables.

Relational databases use SQL (Structured Query Language) for performing INSERT/DELETE/UPDATE operations on the database tables.

However, implementation of SQL varies from one type of database to other.

This raises incompatibility issues.

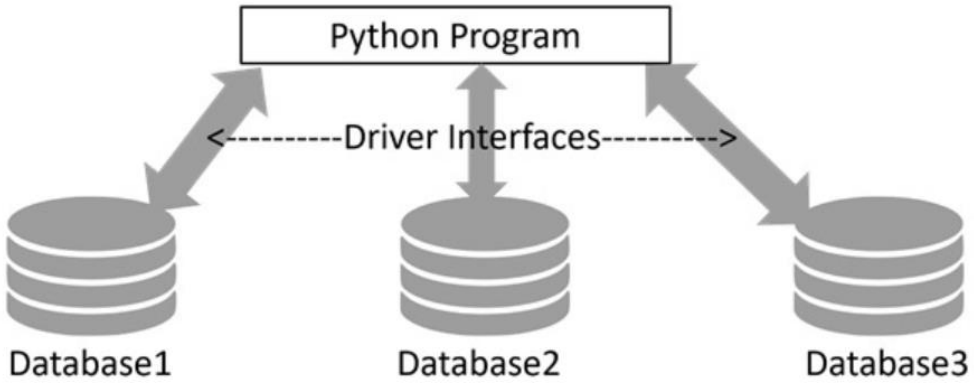
SQL instructions for one database do not match with other.

DB-API (Database API)

To address this issue of compatibility, Python Enhancement Proposal (PEP) 249 introduced a standardized interface known as DB-API.

This interface provides a consistent framework for database drivers, ensuring uniform behavior across different database systems.

It simplifies the process of transitioning between various databases by establishing a common set of rules and methods.



Using SQLite with Python

Python's standard library includes **sqlite3** module, a DB_API compatible driver for SQLite3 database.

It serves as a reference implementation for DB-API.

For other types of databases, you will have to install the relevant Python package.

Database	Python Package
Oracle	cx_oracle, pyodbc
SQL Server	pymssql, pyodbc
PostgreSQL	psycopg2
MySQL	MySQL Connector/Python, pymysql

Working with SQLite

Using SQLite with Python is very easy due to the built-in **sqlite3** module. The process involves

Connection Establishment −

Create a connection object using sqlite3.connect(), providing necessary connection credentials such as server name, port, username, and password.

Transaction Management −

The connection object manages database operations, including opening, closing, and transaction control (committing or rolling back transactions).

Cursor Object −

Obtain a cursor object from the connection to execute SQL queries. The cursor serves as the gateway for CRUD (Create, Read, Update, Delete) operations on the database.

The sqlite3 Module

SQLite is a server-less, file-based lightweight transactional relational database. It doesn't require any installation and no credentials such as username and password are needed to access the database.

Python's sqlite3 module contains DB-API implementation for SQLite database. It is written by Gerhard Häring. Let us learn how to use sqlite3 module for database access with Python.

Let us start by importing sqlite3 and check its version.

```
>>> import sqlite3

>>> sqlite3.sqlite_version

'3.39.4'
```

The Connection Object

A connection object is set up by connect() function in sqlite3 module.

First positional argument to this function is a string representing path (relative or absolute) to a SQLite database file.

The function returns a connection object referring to the database.

>>> conn=sqlite3.connect('testdb.sqlite3')

>>> type(conn)

<class 'sqlite3.Connection'>

Various methods are defined in connection class.

One of them is cursor() method that returns a cursor object.

Transaction control is achieved by commit() and rollback() methods of connection object.

Connection class has important methods to define custom functions and aggregates to be used in SQL queries.

The Cursor Object

Next, we need to get the cursor object from the connection object.

It is the handle to the database when performing any CRUD operation on the database.

The cursor() method on connection object returns the cursor object.

```
>>> cur=conn.cursor()
```

```
>>> type(cur)
```

```
<class 'sqlite3.Cursor'>
```

We can now perform all SQL query operations, with the help of its execute() method available to cursor object.

This method needs a string argument which must be a valid SQL statement.

Creating a Database Table

We shall now add Employee table in our newly created 'testdb.sqlite3' database.

In following script, we call execute() method of cursor object, giving it a string with CREATE TABLE statement inside.

```
import sqlite3

conn=sqlite3.connect('testdb.sqlite3')

cur=conn.cursor()

qry=''' CREATE TABLE Employee (

EmpID INTEGER PRIMARY KEY AUTOINCREMENT, FIRST_NAME TEXT (20), LAST_NAME TEXT(20),

AGE INTEGER, SEX TEXT(1), INCOME FLOAT ); '''

try:

   cur.execute(qry)

   print ('Table created successfully')

except:

   print ('error in creating table')

conn.close()
```

When the above program is run, the database with Employee table is created in the current working directory.

We can verify by listing out tables in this database in SQLite console.

```
sqlite> .open mydb.sqlite
sqlite> .tables
Employee
```

INSERT Operation

The INSERT Operation is required when you want to create your records into a database table.

The following example, executes SQL INSERT statement to create a record in the EMPLOYEE table −

```
import sqlite3

conn=sqlite3.connect('testdb.sqlite3')

cur=conn.cursor()

qry="""INSERT INTO EMPLOYEE(FIRST_NAME,LAST_NAME, AGE, SEX, INCOME) VALUES ('Mac', 'Mohan', 20, 'M', 2000)"""

try:
   cur.execute(qry)
   conn.commit()
   print ('Record inserted successfully')
except:
   conn.rollback()
print ('error in INSERT operation')
```

Insert - Another method

You can also use the parameter substitution technique to execute the INSERT query as follows −

```
import sqlite3
conn=sqlite3.connect('testdb.sqlite3')
cur=conn.cursor()
qry="""INSERT INTO EMPLOYEE(FIRST_NAME,
   LAST_NAME, AGE, SEX, INCOME)
   VALUES (?, ?, ?, ?, ?)"""
try:
   cur.execute(qry, ('Makrand', 'Mohan', 21, 'M', 5000))
   conn.commit()
   print ('Record inserted successfully')
except Exception as e:
   conn.rollback()
   print ('error in INSERT operation')
conn.close()
```

READ Operation

READ Operation on any database means to fetch some useful information from the database.

Once the database connection is established, you are ready to make a query into this database.

You can use either fetchone() method to fetch a single record or fetchall() method to fetch multiple values from a database table.

- **fetchone()** − It fetches the next row of a query result set. A result set is an object that is returned when a cursor object is used to query a table.
- **fetchall()** − It fetches all the rows in a result set. If some rows have already been extracted from the result set, then it retrieves the remaining rows from the result set.
- **rowcount** − This is a read-only attribute and returns the number of rows that were affected by an execute() method.

Example

In the following code, the cursor object executes SELECT * FROM EMPLOYEE query. The resultset is obtained with fetchall() method. We print all the records in the resultset with a **for** loop.

```
import sqlite3

conn=sqlite3.connect('testdb.sqlite3')

cur=conn.cursor()

qry="SELECT * FROM EMPLOYEE"

try:

   # Execute the SQL command

   cur.execute(qry)

   # Fetch all the rows in a list of lists.

   results = cur.fetchall()

   for row in results:

      fname = row[1]

      lname = row[2]
```

```
      sex = row[4]
      income = row[5]
      # Now print fetched result
      print ("fname={},lname={},age={},sex={},income={}".format(fname, lname, age,
sex, income ))
except Exception as e:
   print (e)
   print ("Error: unable to fecth data")

conn.close()
```

It will produce the following **output** –

```
fname=Mac,lname=Mohan,age=20,sex=M,income=2000.0
   fname=Makrand,lname=Mohan,age=21,sex=M,income=5000.0
```

Update Operation

UPDATE Operation on any database means to update one or more records, which are already available in the database.

The following procedure updates all the records having income=2000. Here, we increase the income by 1000.

```
import sqlite3

conn=sqlite3.connect('testdb.sqlite3')

cur=conn.cursor()

qry="UPDATE EMPLOYEE SET INCOME = INCOME+1000 WHERE INCOME=?"

try:
   # Execute the SQL command
   cur.execute(qry, (1000,))
   # Fetch all the rows in a list of lists.
   conn.commit()
   print ("Records updated")
except Exception as e:
   print ("Error: unable to update data")
conn.close()
```

DELETE Operation

DELETE operation is required when you want to delete some records from your database. Following is the procedure to delete all the records from EMPLOYEE where INCOME is less than 2000.

```
import sqlite3

conn=sqlite3.connect('testdb.sqlite3')

cur=conn.cursor()

qry="DELETE FROM EMPLOYEE WHERE INCOME<?"

try:

   # Execute the SQL command

   cur.execute(qry, (2000,))

   # Fetch all the rows in a list of lists.

   conn.commit()

   print ("Records deleted")

except Exception as e:

   print ("Error: unable to delete data")

conn.close()
```

MySQL Database Connection

The PyMySQL Module

PyMySQL is an interface for connecting to a MySQL database server from Python. It implements the Python Database API v2.0 and contains a pure-Python MySQL client library. The goal of PyMySQL is to be a drop-in replacement for MySQLdb.

Installing PyMySQL

```
pip install PyMySQL
```

Note − Make sure you have root privilege to install the above module.

MySQL Database Connection

Before connecting to a MySQL database, make sure of the following points −

- You have created a database TESTDB.
- You have created a table EMPLOYEE in TESTDB.
- This table has fields FIRST_NAME, LAST_NAME, AGE, SEX and INCOME.
- User ID "testuser" and password "test123" are set to access TESTDB.
- Python module PyMySQL is installed properly on your machine.

Example

To use MySQL database instead of SQLite database in earlier examples, we need to change the connect() function as follows −

```
import PyMySQL
# Open database connection
db = PyMySQL.connect("localhost","testuser","test123","TESTDB" )
```

Data Manipulation in Python using Pandas

In Machine Learning, the model requires a dataset to operate, i.e. to train and test.

But data doesn’t come fully prepared and ready to use.

There are discrepancies like Nan/ Null / NA values in many rows and columns.

Sometimes the data set also contains some of the rows and columns which are not even required in the operation of our model.

In such conditions, it requires proper cleaning and modification of the data set to make it an efficient input for our model.

This is achieved by practicing **Data Wrangling** before giving data input to the model.

Creating Sample DataFrame

```
# Importing the pandas library

import pandas as pd

# creating a dataframe object

student_register = pd.DataFrame()

# assigning values to the

# rows and columns of the dataframe

student_register['Name'] = ['Abhijit','Smriti','Akash', 'Roshni']

student_register['Age'] = [20, 19, 20, 14]

student_register['Student'] = [False, True,True, False]

print(student_register)
```

Output:

```
      Name  Age  Student
0  Abhijit   20    False
1   Smriti   19     True
2    Akash   20     True
3   Roshni   14    False
```

As you can see, the dataframe object has four rows [0, 1, 2, 3] and

three columns[“Name”, “Age”, “Student”] respectively.

The column which contains the index values i.e. [0, 1, 2, 3] is known as the **index column**, which is a default part in pandas datagram.

We can change that as per our requirement too because Python is powerful.

Adding data in DataFrame using Append Function

To add a new student in the datagram, i.e you want to add a new row to your existing data frame, that can be achieved by the following code snippet.

One important concept is that the “dataframe” object of Python, consists of rows which are “series” objects instead, stack together to form a table.

Hence adding a new row means creating a new series object and appending it to the dataframe.

```
# creating a new pandas

# series object

new_person = pd.Series(['Mansi', 19, True],

index = ['Name', 'Age', 'Student'])

# using the .append() function # to add that row to the dataframe

student_register.append(new_person, ignore_index = True)

print(student_register)
```

Output:

```
      Name  Age  Student
0  Abhijit   20    False
1   Smriti   19     True
2    Akash   20     True
3   Roshni   14    False
```

Data Manipulation on Dataset

How to perform dataframe creation and data addition in a big dataset.

To exact information of each column, i.e. what type of value it stores and how many of them are unique. There are three support functions, .**shape**, .**info**() and .**corr**() which output the shape of the table, information on rows and columns, and correlation between numerical columns.

```
# dimension of the dataframe

print('Shape: ')

print(student_register.shape)

# showing info about the data

print('Info: ')

print(student_register.info())

# correlation between columns

print('Correlation: ')

print(student_register.corr())
```

```
Shape:
(4, 3)
Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype
---  ------   --------------  -----
0   Name     4 non-null      object
1   Age      4 non-null      int64
2   Student  4 non-null      bool
dtypes: bool(1), int64(1), object(1)
memory usage: 196.0+ bytes
None
Correlation:
             Age   Student
```

In the above example, the .**shape** function gives an output (4, 3) as that is the size of the created dataframe.

The description of the output given by .info() method is as follows:

1. **RangeIndex** describes about the index column, i.e. [0, 1, 2, 3] in our datagram. Which is the number of rows in our dataframe.
2. As the name suggests **Data columns** give the total number of columns as output.
3. **Name, Age, Student** are the name of the columns in our data, non-null tells us that in the corresponding column, there is no NA/ Nan/ None value exists. **object**, **int64** and **bool** are the datatypes each column have.
4. **dtype** gives you an overview of how many data types present in the datagram, which in term simplifies the data cleaning process.

   Also, in high-end machine learning models, **memory usage** is an important term, we can't neglect that.

Getting Statistical Analysis of Data

Before processing and wrangling any data you need to get the overview of it, which includes statistical conclusions like **standard deviation(std), mean and it's quartile distributions**.

for showing the statistical

info of the dataframe

print('Describe')

print(student_register.describe())

Output:

```
Describe

             Age

count   4.000000

mean   18.250000

std     2.872281

min    14.000000

25%    17.750000

50%    19.500000

75%    20.000000

   max    20.000000
```

The description of the output given by .describe() method is as follows:

1. **count** is the number of rows in the dataframe.
2. **mean** is the mean value of all the entries in the “Age” column.
3. **std** is the standard deviation of the corresponding column.
4. **min** and max are the minimum and maximum entry in the column respectively.
5. 25%, 50% and 75% are the **First Quartiles**, **Second Quartile(Median)** and **Third Quartile** respectively, which gives us important info on the distribution of the dataset and makes it simpler to apply an ML model.

Dropping Columns from Data

Drop a column from the data. We will use the drop function from the pandas. We will keep axis = 1 for columns.

```
students = student_register.drop('Age', axis=1)

print(students.head())
```

Output:

```
     Name  Student
0  Abhijit    False
1   Smriti     True
2    Akash     True
3   Roshni    False
```

From the output, we can see that the ‘Age’ column is dropped.

Dropping Rows from Data

To Drop a row from the dataset, use drop function. We will keep axis=0.

```
students = students.drop(2, axis=0)

print(students.head())
```

Output:

```
    Name  Student
0  Abhijit    False
1   Smriti     True
3   Roshni    False
```

In the output we can see that the 2 row is dropped.

Exploratory data analysis

Exploratory data analysis (EDA) is used by data scientists to analyze and investigate data sets and summarize their main characteristics, often employing data visualization methods.
EDA helps determine how best to manipulate data sources to get the answers you need, making it easier for data scientists to discover patterns, spot anomalies, test a hypothesis, or check assumptions.

EDA is primarily used to see what data can reveal beyond the formal modeling or hypothesis testing task and provides a provides a better understanding of data set variables and the relationships between them.

It can also help determine if the statistical techniques you are considering for data analysis are appropriate.

Originally developed by American mathematician John Tukey in the 1970s, EDA techniques continue to be a widely used method in the data discovery process today.

Why is exploratory data analysis important in data science?

The main purpose of EDA is to help look at data before making any assumptions.

It can help identify obvious errors, as well as better understand patterns within the data, detect outliers or anomalous events, find interesting relations among the variables.

Data scientists can use exploratory analysis to ensure the results they produce are valid and applicable to any desired business outcomes and goals.

EDA also helps stakeholders by confirming they are asking the right questions.

EDA can help answer questions about standard deviations, categorical variables, and confidence intervals.

Once EDA is complete and insights are drawn, its features can then be used for more sophisticated data analysis or modeling, including machine learning.

Exploratory data analysis tools

Specific statistical functions and techniques you can perform with EDA tools include:

- Clustering and dimension reduction techniques, which help create graphical displays of high-dimensional data containing many variables.
- Univariate visualization of each field in the raw dataset, with summary statistics.
- Bivariate visualizations and summary statistics that allow you to assess the relationship between each variable in the dataset and the target variable you're looking at.
- Multivariate visualizations, for mapping and understanding interactions between different fields in the data.
- K-means Clustering is a clustering method in unsupervised learning where data points are assigned into K groups, i.e. the number of clusters, based on the distance from each group's centroid. The data points closest to a particular centroid will be clustered under the same category. K-means Clustering is commonly used in market segmentation, pattern recognition, and image compression.
- Predictive models, such as linear regression, use statistics and data to predict outcomes.

Types of exploratory data analysis

There are four primary types of EDA:

1. **Univariate non-graphical.** This is simplest form of data analysis, where the data being analyzed consists of just one variable. Since it's a single variable, it doesn't deal with causes or relationships. The main purpose of univariate analysis is to describe the data and find patterns that exist within it.

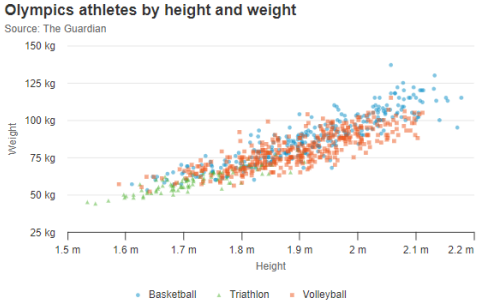
1. **Univariate graphical**. Non-graphical methods don't provide a full picture of the data. Graphical methods are therefore required. Common types of univariate graphics include:
   - Stem-and-leaf plots, which show all data values and the shape of the distribution.
   - Histograms, a bar plot in which each bar represents the frequency (count) or proportion (count/total count) of cases for a range of values.
   - Box plots, which graphically depict the five-number summary of minimum, first quartile, median, third quartile, and maximum.

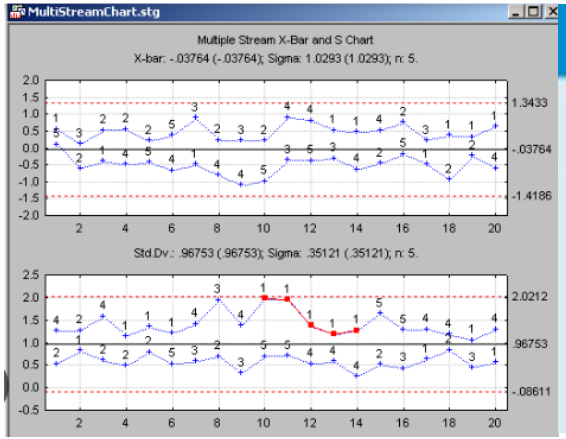
3. Multivariate nongraphical: Multivariate data arises from more than one variable. Multivariate non-graphical EDA techniques generally show the relationship between two or more variables of the data through cross-tabulation or statistics.

4. Multivariate graphical: Multivariate data uses graphics to display relationships between two or more sets of data. The most used graphic is a grouped bar plot or bar chart with each group representing one level of one of the variables and each bar within a group representing the levels of the other variable.

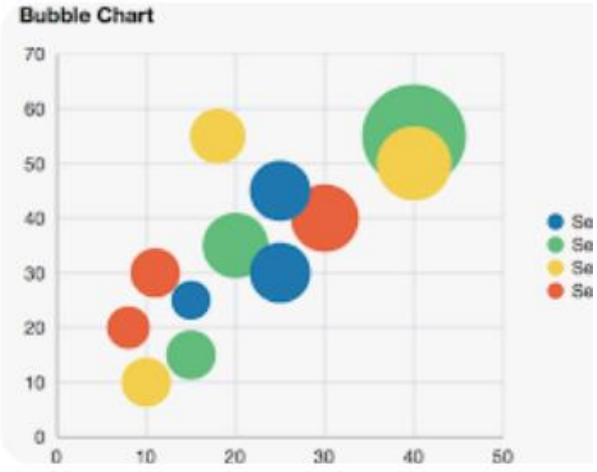
Other common types of multivariate graphics include:

- Scatter plot, which is used to plot data points on a horizontal and a vertical axis to show how much one variable is affected by another.
- Multivariate chart, which is a graphical representation of the relationships between factors and a response.
- Run chart, which is a line graph of data plotted over time.
- Bubble chart, which is a data visualization that displays multiple circles (bubbles) in a two-dimensional plot.
- Heat map, which is a graphical representation of data where values are depicted by color.


Olympics athletes by height and weight
Source: The Guardian
150 kg
125 kg
100 kg
75 kg
50 kg
25 kg
Weight
1.5 m
1.6 m
1.7 m
1.8 m
1.9 m
2 m
2.1 m
2.2 m
Height
Basketball
Triathlon
Volleyball


MultiStreamChart.stg
Multiple Stream X-Bar and S Chart
X-bar: -.03764 (-.03764); Sigma: 1.0293 (1.0293); n: 5.
1.3433
-.03764
-1.4186
Std.Dv.: .96753 (.96753); Sigma: .35121 (.35121); n: 5.
2.0212
.96753
-.08611


Run Chart
A Run Chart is a time series plot that displays shifts and trends in data over time.
Value
Time


Bubble Chart
70
60
50
40
30
20
10
0
0
10
20
30
40
50


National Digital Forecast Database

Exploratory Data Analysis Tools

Most common data science tools used to create an EDA include:

Python:

- An interpreted, object-oriented programming language with dynamic semantics.
- Its high-level, built-in data structures, combined with dynamic typing and dynamic binding, make it very attractive for rapid application development, as well as for use as a scripting or glue language to connect existing components together.
- Python and EDA can be used together to identify missing values in a data set, which is important so you can decide how to handle missing values for machine learning.

R:

- An open-source programming language and free software environment for statistical computing and graphics supported by the R Foundation for Statistical Computing.
- The R language is widely used among statisticians in data science in developing statistical observations and data analysis.

Descriptive Statistics

Whenever we deal with some piece of data no matter whether it is small or stored in huge databases statistics is the key that helps us to analyze this data and provide insightful points to understand the whole data without going through each of the data pieces in the complete dataset at hand.

In Descriptive statistics, we are describing our data with the help of various representative methods using charts, graphs, tables, excel files, etc.

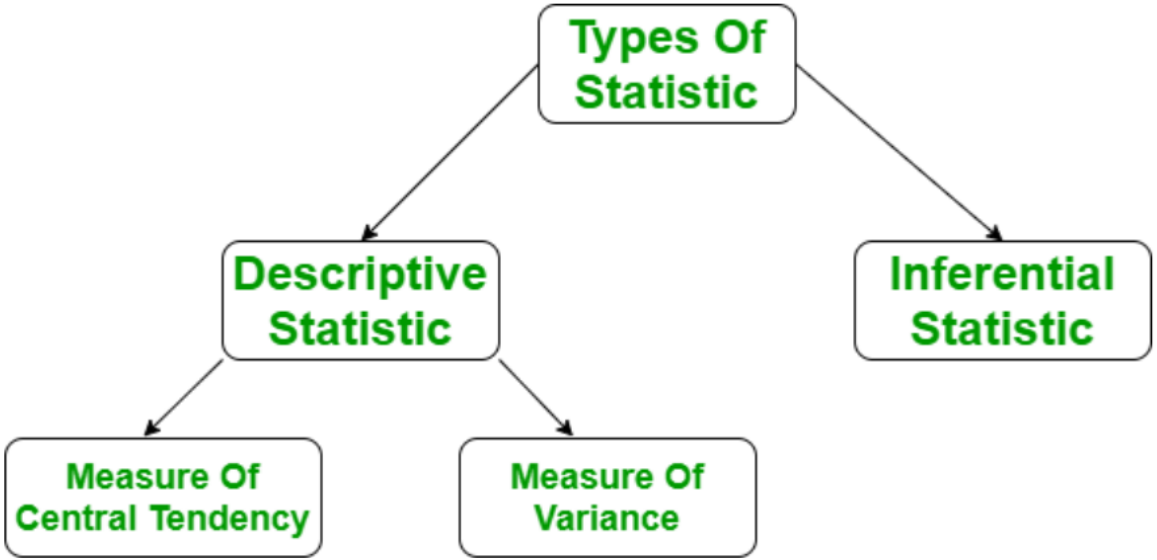
In descriptive statistics, we describe our data in some manner and present it in a meaningful way so that it can be easily understood.

Most of the time it is performed on small data sets and this analysis helps us a lot to predict some future trends based on the current findings.

Some measures that are used to describe a data set are measures of central tendency and measures of variability or dispersion.

Types of Descriptive Statistics

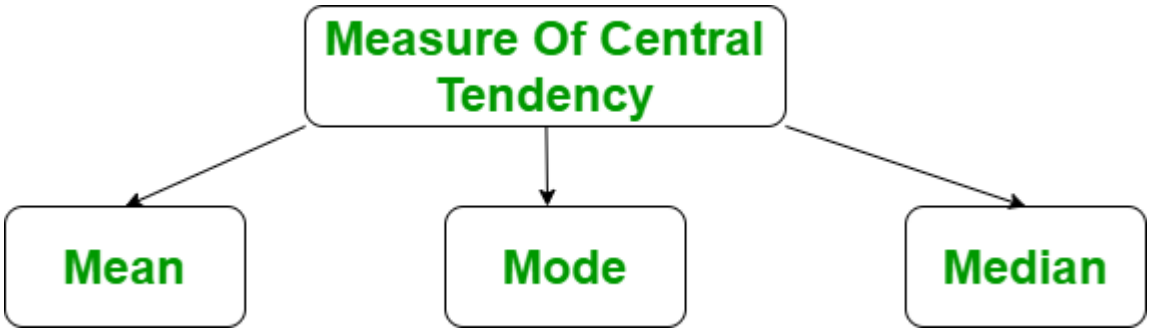
- Measures of Central Tendency
- Measure of Variability
- Measures of Frequency Distribution



Measures of Central Tendency

It represents the whole set of data by a single value. It gives us the location of the central points. There are three main measures of central tendency:

- Mean
- Mode
- Median



Mean

It is the sum of observations divided by the total number of observations. It is also defined as average which is the sum divided by count.

$$\bar{x} = \frac{\sum x}{n}$$

where,

- x = Observations
- n = number of terms

```
import numpy as np # Sample Data arr = [5, 6, 11]

# Mean mean = np.mean(arr)

print("Mean = ", mean)
```

Output :

Mean = 7.333333333333333

Mode

It is the value that has the highest frequency in the given data set. The data set may have no mode if the frequency of all data points is the same.

Also, we can have more than one mode if we encounter two or more data points having the same frequency.

```
from scipy import stats

# sample Data arr = [1, 2, 2, 3]

# Mode mode = stats.mode(arr)

print("Mode = ", mode)
```

Output:

```
Mode = ModeResult(mode=array([2]), count=array([2]))
```

Median

It is the middle value of the data set.

It splits the data into two halves.

If the number of elements in the data set is odd then the center element is the median and if it is even then the median would be the average of two central elements.

```
import numpy as np

# sample Data arr = [1, 2, 3, 4]

# Median median = np.median(arr)

print("Median = ", median)
```

Output:

Median = 2.5

Measure of Variability

Measures of variability are also termed measures of dispersion as it helps to gain insights about the dispersion or the spread of the observations at hand.

Some of the measures which are used to calculate the measures of dispersion in the observations of the variables are as follows:

- Range
- Variance
- Standard deviation

Range

The range describes the difference between the largest and smallest data point in our data set. The bigger the range, the more the spread of data and vice versa.

Range = Largest data value – smallest data value

```
import numpy as np

# Sample Data arr = [1, 2, 3, 4, 5]

# Finding Max Maximum = max(arr)

# Finding Min Minimum = min(arr)

# Difference Of Max and Min Range = Maximum-Minimum

print("Maximum = {}, Minimum = {} and Range = {}".format( Maximum, Minimum, Range))
```

Variance

It is defined as an average squared deviation from the mean. It is calculated by finding the difference between every data point and the average which is also known as the mean, squaring them, adding all of them, and then dividing by the number of data points present in our data set.

$$\sigma^2 = \frac{\sum(x-\mu)^2}{N}$$

where,

- **x** -> Observation under consideration
- **N** -> number of terms
- **mu** -> Mean

```
import statistics
# sample data arr = [1, 2, 3, 4, 5]
# variance print("Var = ", (statistics.variance(arr)))
```

Output:

Var = 2.5

Standard Deviation

It is defined as the square root of the variance. It is calculated by finding the Mean, then subtracting each number from the Mean which is also known as the average, and squaring the result. Adding all the values and then dividing by the no of terms followed by the square root.

where,

$$\sigma = \sqrt{\frac{\sum(x-\mu)^2}{N}}$$

- x = Observation under consideration
- N = number of terms
- mu = Mean

```
import statistics

# sample data arr = [1, 2, 3, 4, 5]

# Standard Deviation print("Std = ", (statistics.stdev(arr)))
```

Output:

Std = 1.5811388300841898

Measures of Frequency Distribution

Measures of frequency distribution help us gain valuable insights into the distribution and the characteristics of the dataset.

Measures like,

- Count
- Frequency
- Relative Frequency
- Cumulative Frequency

are used to analyze the dataset on the basis of measures of frequency distribution.

Univariate v/s Bivariate

While performing the data analysis if we are considering only one variable to gain insights about it then it is known as the **univariate data analysis.**

But if we are trying to gain insights into one variable with respect to some other variable like calculating correlation or covariance between two variables then this is known as data analysis.

Bivariate analysis helps us establish a relationship between two variables which helps us make better decisions so, that we can manipulate one to cause a positive or negative effect on the other based on the relationship between the two variables.

Even though we can use multivariate data analysis to analyze and derive relationships between more than two variables these methodologies are considered to come under the domain of machine learning.

Descriptive Statistics v/s Inferential Statistics

Generally, there are two types of statistics that are used to deal with the data when the requirements of the analyst are different.

The main difference lies in the final requirements only if the person just wants to extract meaningful insights from the data at hand then the domain of statistics that will be used by him is known as Descriptive Statistics

If we would like to use the observations to predict the future data let's say in the time series related dataset then our objective contains the process of inference as well and hence it is also known as Inferential Statistics.

We can summarize this as when we would like to make some predictions/inferences based on some dataset at hand then those statistical methods are known as inferential statistics.

Data Visualization using Matplotlib in Python

Data visualization is a crucial aspect of data analysis, enabling data scientists and analysts to present complex data in a more understandable and insightful manner.

One of the most popular libraries for data visualization in Python is **Matplotlib**.

Let us learn the usage of Matplotlib for creating various types of plots and customizing them to fit specific needs and how to visualize data with the help of the Matplotlib library of Python.

Matplotlib is a versatile and widely-used data visualization library in Python. It allows users to create static, interactive, and animated visualizations.

The library is built on top of NumPy, making it efficient for handling large datasets.

This library is built on the top of NumPy arrays and consist of several plots like line chart, bar chart, histogram, etc. It provides a lot of flexibility but at the cost of writing more code.

Installing Matplotlib for Data Visualization

pip install matplotlib

We can also install matplotlib in Jupyter Notebook and Google colab environment by using the same command:

If you are using **Jupyter Notebook**, you can install it within a notebook cell by using:

!pip install matplotlib

Matplotlib provides a **module called pyplot,** which offers a **MATLAB-like interface for creating plots and charts.**

It simplifies the process of generating various types of visualizations by offering a collection of functions that handle common plotting tasks.

Each function in Pyplot modifies a figure in a specific way.

Key Pyplot Functi

Category	Function	Description
Plot Creation	plot()	Creates line plots with customizable styles.
	scatter()	Generates scatter plots to visualize relationships.
Graphical Elements	bar()	Creates bar charts for comparing categories.
	hist()	Draws histograms to show data distribution.
	pie()	Creates pie charts to represent parts of a whole.

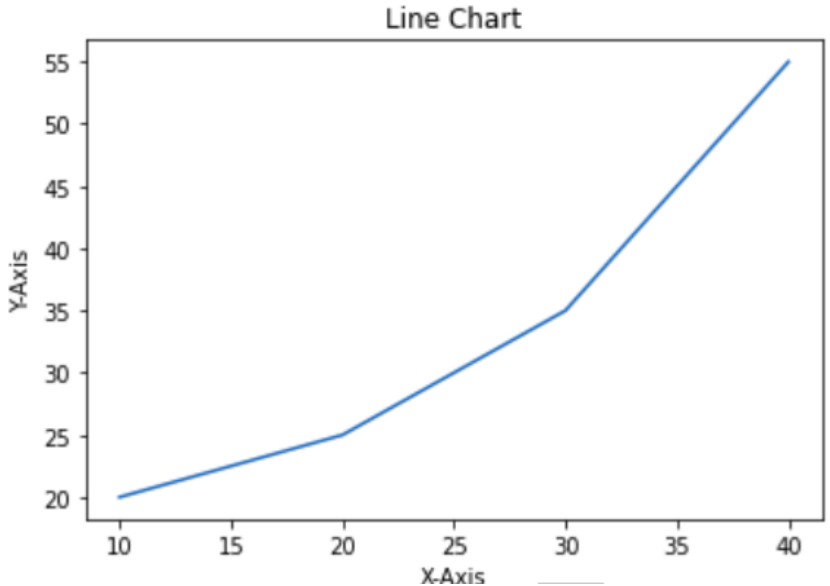
Customization	xlabel(), ylabel()	Sets labels for the X and Y axes.
	title()	Adds a title to the plot.
	legend()	Adds a legend to differentiate data series.
Visualization Control	xlim(), ylim()	Sets limits for the X and Y axes.
	grid()	Adds gridlines to the plot for readability.
	show()	Displays the plot in a window.

Figure Management	figure()	Creates or activates a figure.
	subplot()	Creates a grid of subplots within a figure.
	savefig()	Saves the current figure to a file.

Line Chart

```
import matplotlib.pyplot as plt


# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

# plotting the data
plt.plot(x, y)

# Adding title to the plot
plt.title("Line Chart")

# Adding label on the y-axis
plt.ylabel('Y-Axis')

# Adding label on the x-axis
plt.xlabel('X-Axis')

plt.show()
```

Output:



Bar Chart

A bar chart is a graph that represents the category of data with rectangular bars with lengths and heights that is proportional to the values which they represent.

The bar plots can be plotted horizontally or vertically.

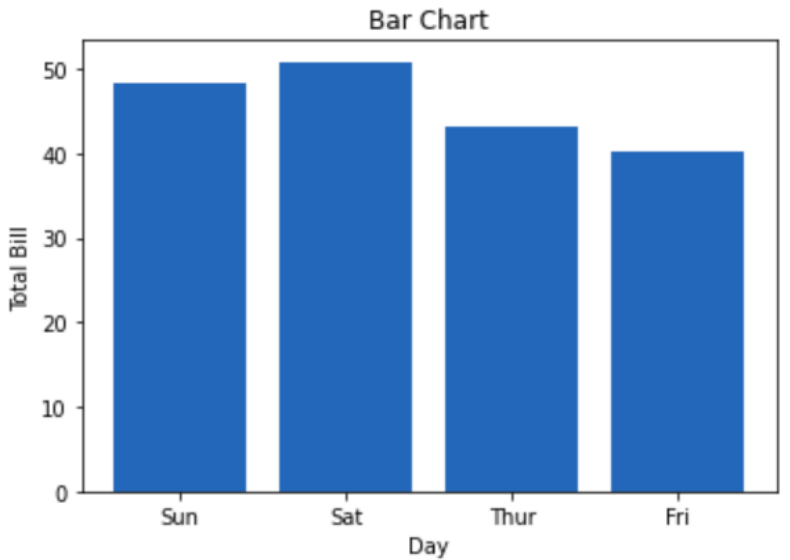
A bar chart describes the comparisons between the discrete categories.

It can be created using the bar() method.

example, we will use the tips dataset. Tips database is the record of the tip given by the customers in a restaurant for two and a half months in the early 1990s. It contains 6 columns as total_bill, tip, sex, smoker, day, time, size.

```
import matplotlib.pyplot as plt
import pandas as pd

# Reading the tips.csv file
data = pd.read_csv('tips.csv')

# initializing the data
x = data['day']
y = data['total_bill']

# plotting the data
plt.bar(x, y)

# Adding title to the plot
plt.title("Tips Dataset")

# Adding label on the y-axis
plt.ylabel('Total Bill')

# Adding label on the x-axis
plt.xlabel('Day')

plt.show()
```

Output:



Histogram

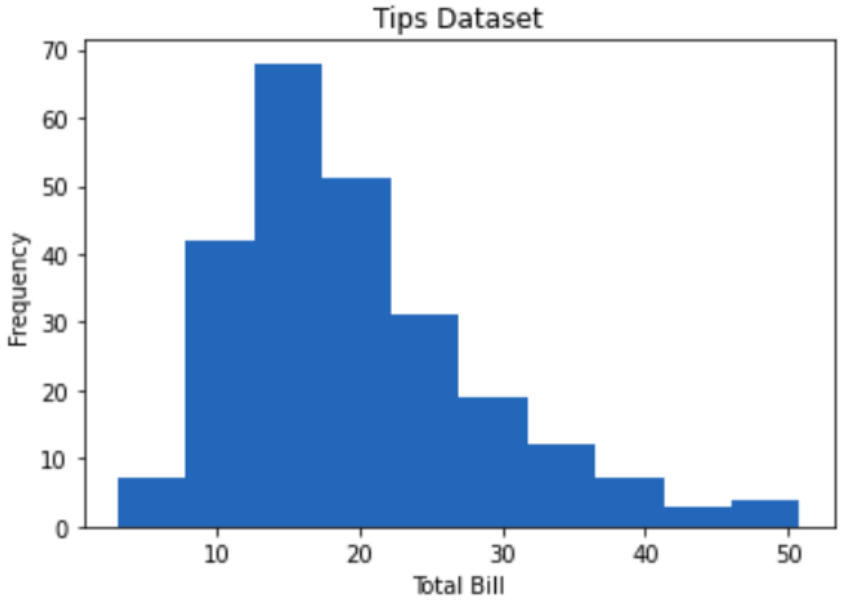
A **histogram** is basically used to represent data provided in a form of some groups.

It is a type of bar plot where the X-axis represents the bin ranges while the Y-axis gives information about frequency.

The **hist()** function is used to compute and create histogram of x.

```
import matplotlib.pyplot as plt
import pandas as pd

# Reading the tips.csv file
data = pd.read_csv('tips.csv')

# initializing the data
x = data['total_bill']

# plotting the data
plt.hist(x)

# Adding title to the plot
plt.title("Tips Dataset")

# Adding label on the y-axis
plt.ylabel('Frequency')

# Adding label on the x-axis
plt.xlabel('Total Bill')

plt.show()
```

Output:



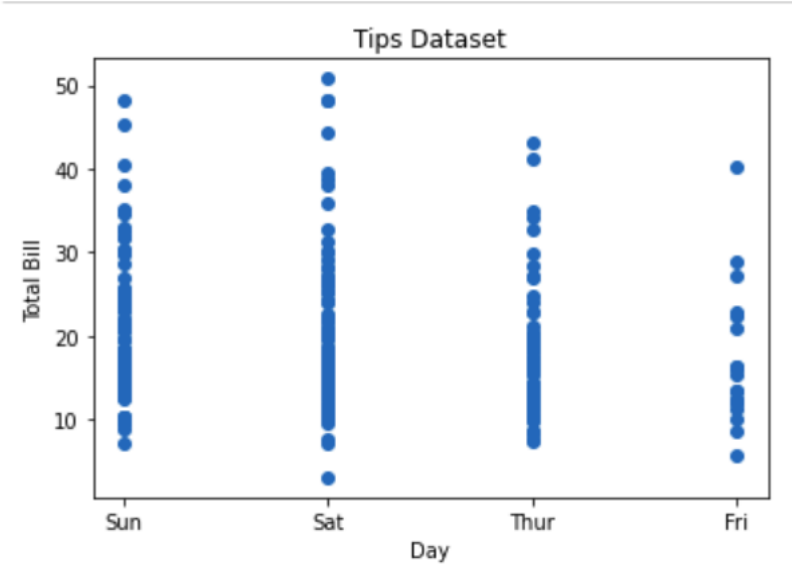
Scatter plot

Scatter plots are used to observe relationships between variables.

The **scatter()** method in the matplotlib library is used to draw a scatter plot.

```
import matplotlib.pyplot as plt
import pandas as pd

# Reading the tips.csv file
data = pd.read_csv('tips.csv')

# initializing the data
x = data['day']
y = data['total_bill']

# plotting the data
plt.scatter(x, y)

# Adding title to the plot
plt.title("Tips Dataset")

# Adding label on the y-axis
plt.ylabel('Total Bill')

# Adding label on the x-axis
plt.xlabel('Day')

plt.show()
```

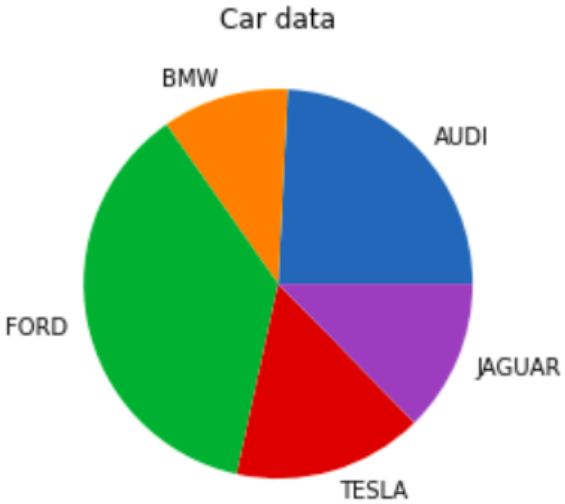
Output:



Pie chart is a circular chart used to display only one series of data. The area of slices of the pie represents the percentage of the parts of the data. The slices of pie are called wedges. It can be created using the pie() method.

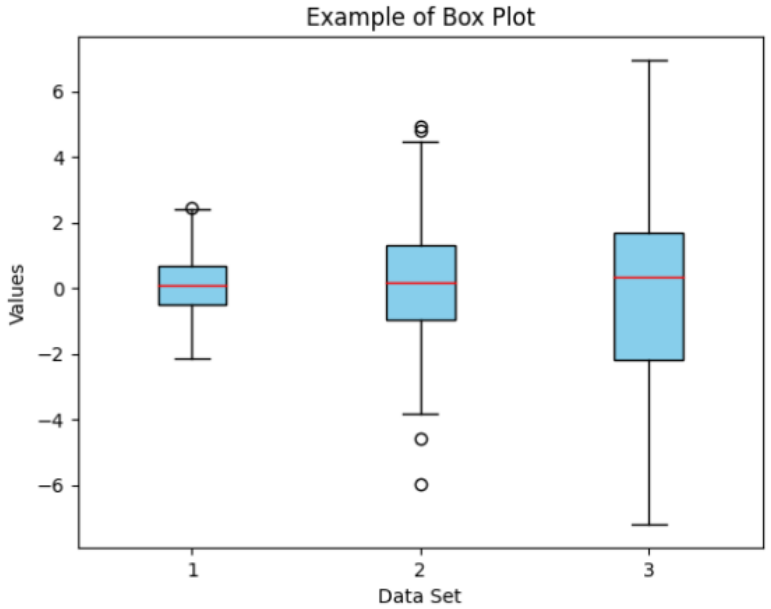
```
import matplotlib.pyplot as plt
import pandas as pd

# Reading the tips.csv file
data = pd.read_csv('tips.csv')

# initializing the data
cars = ['AUDI', 'BMW', 'FORD',
        'TESLA', 'JAGUAR',]
data = [23, 10, 35, 15, 12]

# plotting the data
plt.pie(data, labels=cars)

# Adding title to the plot
plt.title("Car data")

plt.show()
```

Output:



A Box Plot, also known as a Whisker Plot, is a standardized way of displaying the distribution of data based on a five-number summary: minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum. It can also show outliers.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(10)
data = [np.random.normal(0, std, 100) for std in range(1, 4)]

# Create a box plot
plt.boxplot(data, vert=True, patch_artist=True,
            boxprops=dict(facecolor='skyblue'),
            medianprops=dict(color='red'))

plt.xlabel('Data Set')
plt.ylabel('Values')
plt.title('Example of Box Plot')
plt.show()
```



Box Plot in Python using Matplotlib

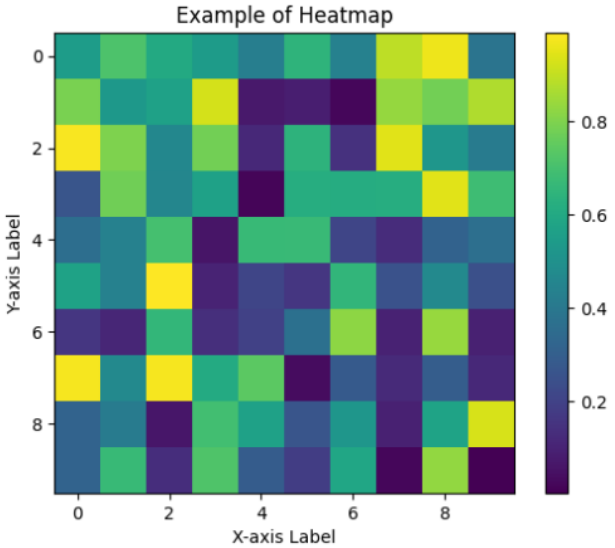
Explanation:

- `plt.boxplot(data)`: Creates the box plot**. The `vert=True` argument makes the plot vertical, and `patch_artist=True` fills the box with color.**
- **`boxprops` and `medianprops`**: Customize the appearance of the boxes and median lines, respectively.

The box shows the interquartile range (IQR), the line inside the box shows the median, and the “whiskers” extend to the minimum and maximum values within 1.5 * IQR from the first and third quartiles. Any points outside this range are considered outliers and are plotted as individual points.

A Heatmap is a data visualization technique that represents data in a matrix form, where individual values are represented as colors. Heatmaps are particularly useful for visualizing the magnitude of data across a two-dimensional surface and identifying patterns, correlations, and concentrations.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)
data = np.random.rand(10, 10)

# Create a heatmap
plt.imshow(data, cmap='viridis', interpolation='nearest')
# Add a color bar to show the scale
plt.colorbar()

plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Example of Heatmap')

plt.show()
```



Heatmap with matplotlib

Explanation:

- **`plt.imshow(data, cmap='viridis')`**: Displays the data as an image (heatmap). The `cmap='viridis'` argument specifies the color map used for the heatmap.
- **`interpolation='nearest'`**: Ensures that each data point is shown as a block of color without smoothing.

The color bar on the side provides a scale to interpret the colors, with darker colors representing lower values and lighter colors representing higher values. This type of plot is often used in fields like data analysis, bioinformatics, and finance to visualize data correlations and distributions across a matrix.

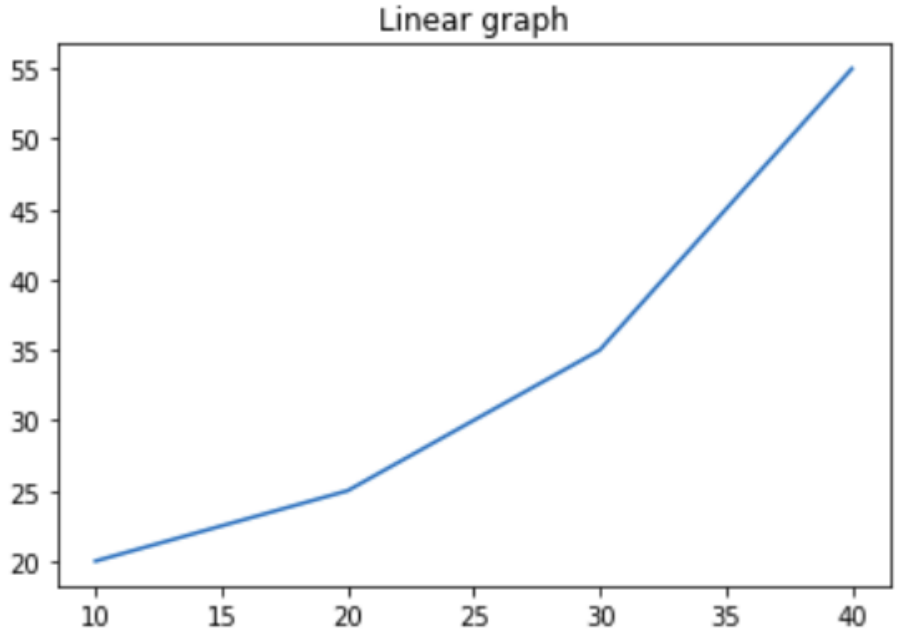
Matplotlib Customization and Styling

Matplotlib allows extensive customization of plots, including changing colors, adding labels, and modifying plot styles.

1. Adding a Title

The title() method in matplotlib module is used to specify the title of the visualization depicted and displays the title using various attributes.

```
import matplotlib.pyplot as plt

# initializing the data

x = [10, 20, 30, 40]

y = [20, 25, 35, 55]


# plotting the data

plt.plot(x, y)


# Adding title to the plot

plt.title("Linear graph")


plt.show()
```



We can also change the appearance of the title by using the parameters of this function.

```
import matplotlib.pyplot as plt

# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

# plotting the data
plt.plot(x, y)

# Adding title to the plot
plt.title("Linear graph", fontsize=25, color="green")

plt.show()
```

2. Adding X Label and Y Label

In layman's terms, the X label and the Y label are the titles given to X-axis and Y-axis respectively. These can be added to the graph by using the **xlabel()** and **ylabel()** methods.

```
# initializing the data

x = [10, 20, 30, 40]

y = [20, 25, 35, 55]

# plotting the data

plt.plot(x, y)

# Adding title to the plot

plt.title("Linear graph", fontsize=25, color="green")

# Adding label on the y-axis

plt.ylabel('Y-Axis')

# Adding label on the x-axis

plt.xlabel('X-Axis')

plt.show()
```

3. Setting Limits and Tick labels You might have seen that Matplotlib automatically sets the values and the markers(points) of the X and Y axis, however, it is possible to set the limit and markers manually.

- **xlim()** and **ylim()** functions are used to set the limits of the X-axis and Y-axis respectively.
- Similarly, **xticks()** and **yticks()** functions are used to set tick labels.

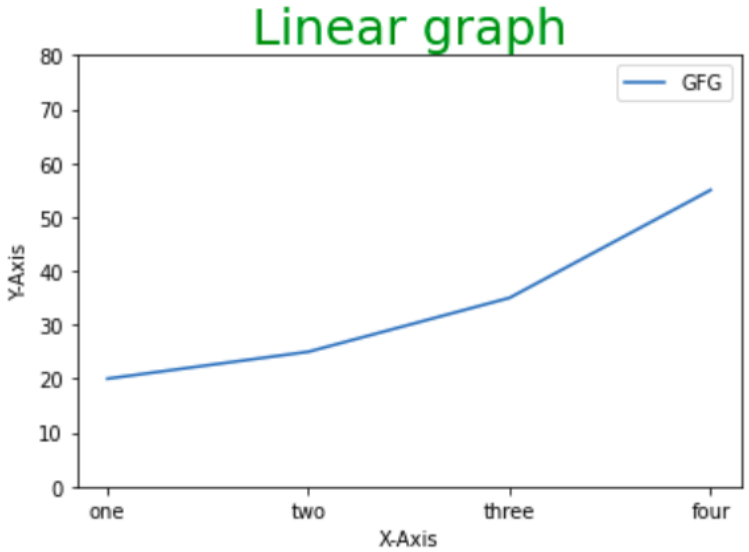
```
x = [10, 20, 30, 40] # initializing the data

y = [20, 25, 35, 55]

plt.plot(x, y)  # plotting the data

plt.title("Linear graph", fontsize=25, color="green") # Adding title to the plot

plt.ylabel('Y-Axis') # Adding label on the y-axis

# Adding label on the x-axis

plt.xlabel('X-Axis')

# Setting the limit of y-axis

plt.ylim(0, 80)

# setting the labels of x-axis

plt.xticks(x, labels=["one", "two", "three", "four"])


plt.show()
```

4. Adding Legends

A legend is an area describing the elements of the graph. In simple terms, it reflects the data displayed in the graph's Y-axis. It generally appears as the box containing a small sample of each color on the graph and a small description of what this data means.

The attribute bbox_to_anchor=(x, y) of legend() function is used to specify the coordinates of the legend, and the attribute ncol represents the number of columns that the legend has. Its default value is 1.

```
plt.xlabel('X-Axis') # Adding label on the x-axis

plt.ylim(0, 80)  # Setting the limit of y-axis

plt.xticks(x, labels=["one", "two", "three", "four"])  # setting the lab

# Adding legends

plt.legend(["GFG"])

plt.show()
```



Customizing Line Chart

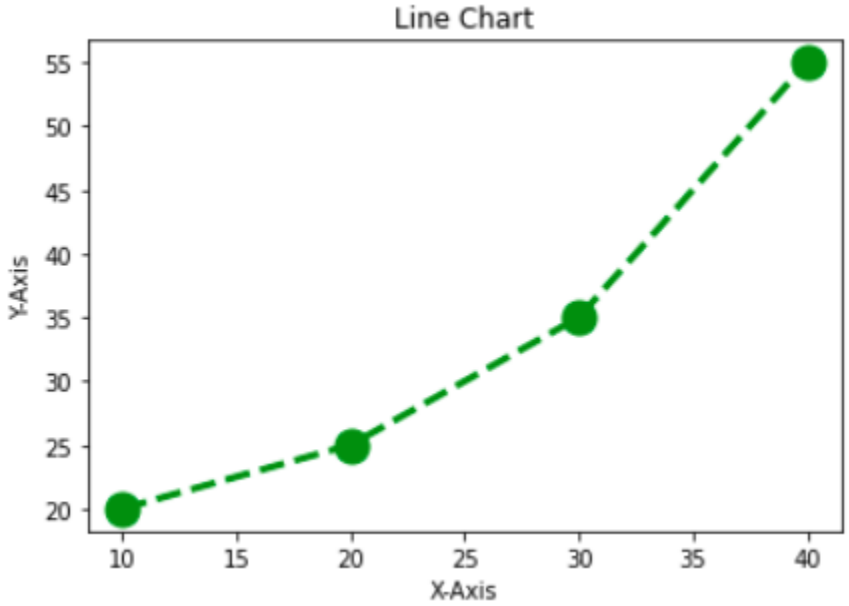
To customize the above-created line chart, We will be using the following properties:

- **color:** Changing the color of the line
- **linewidth:** Customizing the width of the line
- **marker:** For changing the style of actual plotted point
- **markersize:** For changing the size of the markers
- **linestyle:** For defining the style of the plotted line

Different Linestyle available

Character	Definition
–	Solid line
—	Dashed line
-.	dash-dot line
:	Dotted line
.	Point marker
o	Circle marker
,	Pixel marker
v	triangle_down marker
^	triangle_up marker
<	triangle_left marker
>	triangle_right marker
1	tri_down marker
2	tri_up marker
3	tri_left marker
4	tri_right marker
s	square marker
p	pentagon marker
*	star marker
h	hexagon1 marker
H	hexagon2 marker
+	Plus marker
x	X marker
D	Diamond marker
d	thin_diamond marker
|	vline marker
_	hline marker

```
import matplotlib.pyplot as plt


# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

# plotting the data
plt.plot(x, y, color='green', linewidth=3, marker='o',
        markersize=15, linestyle='--')

# Adding title to the plot
plt.title("Line Chart")

# Adding label on the y-axis
plt.ylabel('Y-Axis')

# Adding label on the x-axis
plt.xlabel('X-Axis')

plt.show()
```



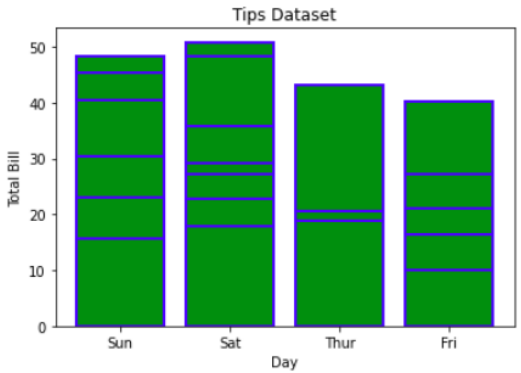
Customizing Bar Chart

To make bar charts more informative and visually appealing, various customization options are available. Customization that is available for the Bar Chart are:

- **color:** For the bar faces
- **edgecolor:** Color of edges of the bar
- **linewidth:** Width of the bar edges
- **width:** Width of the bar

```
import matplotlib.pyplot as plt
import pandas as pd

# Reading the tips.csv file
data = pd.read_csv('tips.csv')

# initializing the data
x = data['day']
y = data['total_bill']

# plotting the data
plt.bar(x, y, color='green', edgecolor='blue',
        linewidth=2)

# Adding title to the plot
plt.title("Tips Dataset")

# Adding label on the y-axis
plt.ylabel('Total Bill')

# Adding label on the x-axis
plt.xlabel('Day')

plt.show()
```

Output:



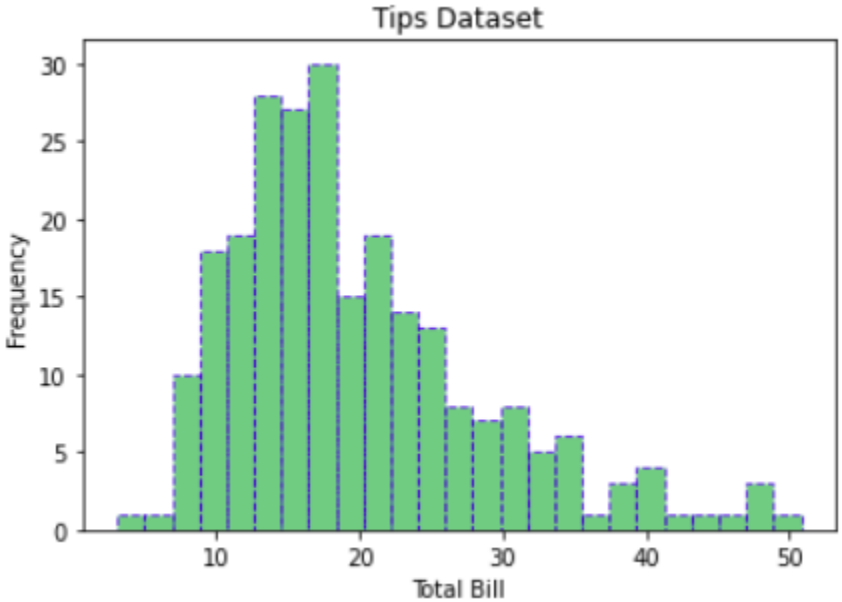
Note: The lines in between the bars refer to the different values in the Y-axis of the particular value of the X-axis.

Customizing Histogram Plot

To make histogram plots more effective and tailored to your data, you can apply various customizations:

- **bins:** Number of equal-width bins
- **color:** For changing the face color
- **edgecolor:** Color of the edges
- **linestyle:** For the edgelines
- **alpha:** blending value, between 0 (transparent) and 1 (opaque)

```
import matplotlib.pyplot as plt
import pandas as pd

# Reading the tips.csv file
data = pd.read_csv('tips.csv')

# initializing the data
x = data['total_bill']

# plotting the data
plt.hist(x, bins=25, color='green', edgecolor='blue',
         linestyle='--', alpha=0.5)

# Adding title to the plot
plt.title("Tips Dataset")

# Adding label on the y-axis
plt.ylabel('Frequency')

# Adding label on the x-axis
plt.xlabel('Total Bill')

plt.show()
```

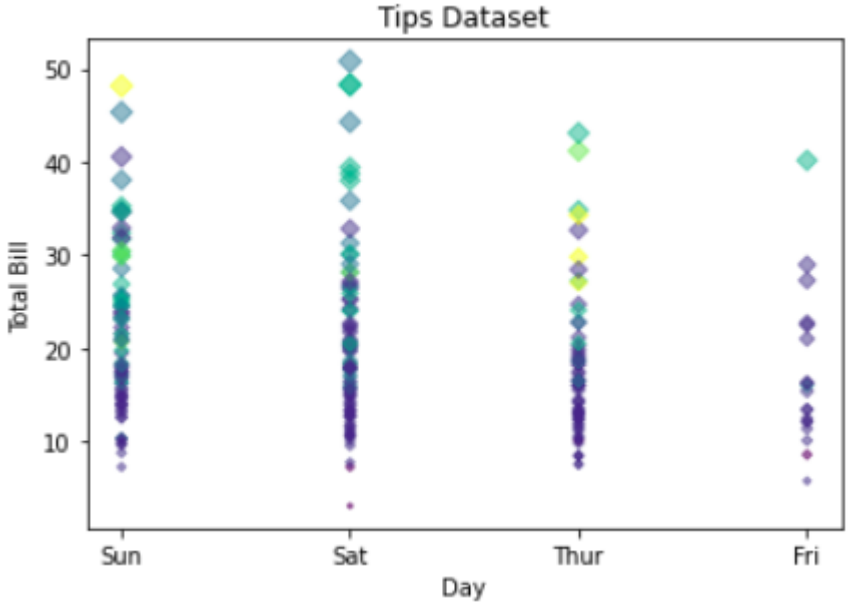


Customizing Scatter Plot

Scatter plots are versatile tools for visualizing relationships between two variables. Customizations that are available for the scatter plot are to enhance their clarity and effectiveness:

- **s:** marker size (can be scalar or array of size equal to size of x or y)
- **c:** color of sequence of colors for markers
- **marker:** marker style
- **linewidths:** width of marker border
- **edgecolor:** marker border color
- **alpha:** blending value, between 0 (transparent) and 1 (opaque)

```
import matplotlib.pyplot as plt
import pandas as pd

# Reading the tips.csv file
data = pd.read_csv('tips.csv')

# initializing the data
x = data['day']
y = data['total_bill']

# plotting the data
plt.scatter(x, y, c=data['size'], s=data['total_bill'],
            marker='D', alpha=0.5)

# Adding title to the plot
plt.title("Tips Dataset")

# Adding label on the y-axis
plt.ylabel('Total Bill')

# Adding label on the x-axis
plt.xlabel('Day')

plt.show()
```

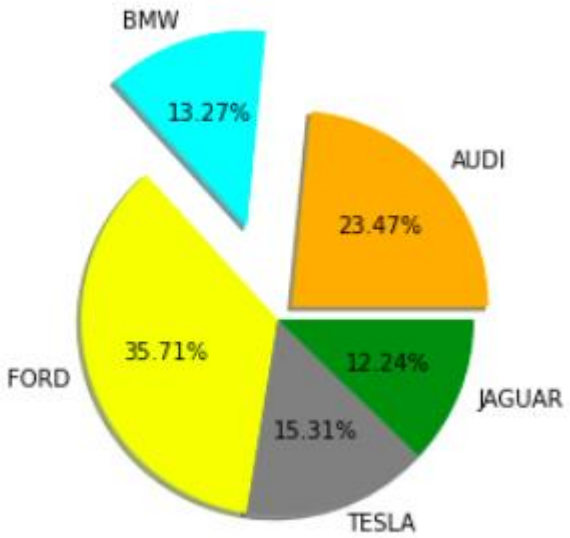


Customizing Pie Chart

Pie charts are a great way to visualize proportions and parts of a whole. To make your pie charts more effective and visually appealing, consider the following customization techniques:

- **explode:** Moving the wedges of the plot
- **autopct:** Label the wedge with their numerical value.
- **color:** Attribute is used to provide color to the wedges.
- **shadow:** Used to create shadow of wedge.

```
import matplotlib.pyplot as plt
import pandas as pd

# Reading the tips.csv file
data = pd.read_csv('tips.csv')

# initializing the data
cars = ['AUDI', 'BMW', 'FORD',
        'TESLA', 'JAGUAR',]
data = [23, 13, 35, 15, 12]

explode = [0.1, 0.5, 0, 0, 0]

colors = ( "orange", "cyan", "yellow",
          "grey", "green",)

# plotting the data
plt.pie(data, labels=cars, explode=explode, autopct='%1.2f%%',
        colors=colors, shadow=True)

plt.show()
```



Matplotlib's Core Components: Figures and Axes

Few important classes are:

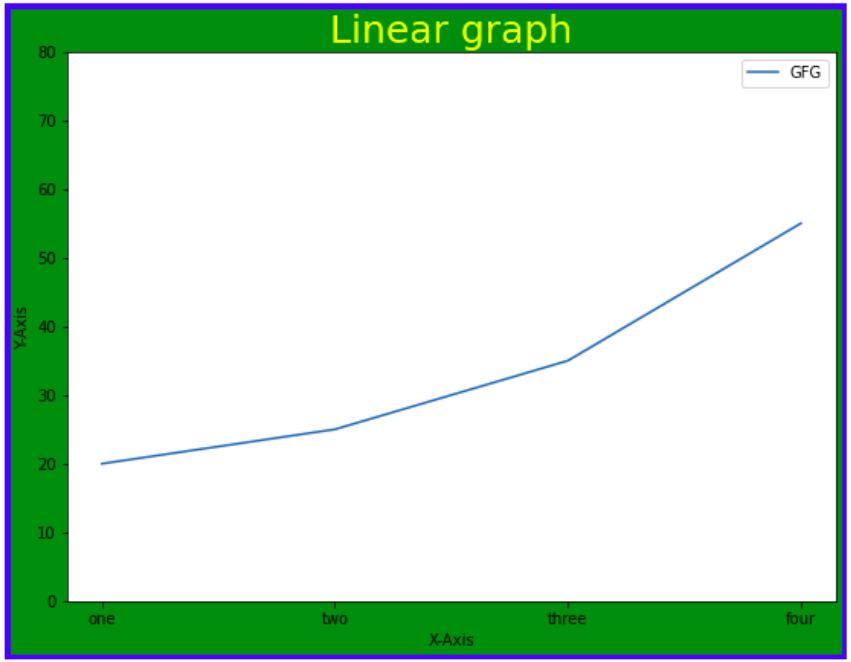
- **Figure**
- **Axes**

Note: Matplotlib take care of the creation of inbuilt defaults like Figure and Axes.

1. Figure class

Consider the figure class as the overall window or page on which everything is drawn. It is a top-level container that contains one or more axes. A figure can be created using the **figure()** method.

```
# Python program to show pyplot module
import matplotlib.pyplot as plt
from matplotlib.figure import Figure

# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

# Creating a new figure with width = 7 inches
# and height = 5 inches with face color as
# green, edgecolor as red and the line width
# of the edge as 7
fig = plt.figure(figsize =(7, 5), facecolor='g',
                 edgecolor='b', linewidth=7)

ax = fig.add_axes([1, 1, 1, 1]) # Creating a new axes for the figure

ax.plot(x, y)  # Adding the data to be plotted

plt.title("Linear graph", fontsize=25, color="yellow")  # Adding title to the plot

plt.ylabel('Y-Axis')   # Adding label on the y-axis

plt.xlabel('X-Axis')  # Adding label on the x-axis

plt.ylim(0, 80)  # Setting the limit of y-axis

plt.xticks(x, labels=["one", "two", "three", "four"])  # setting the labels of x-axis

plt.legend(["GFG"]) # Adding legends

plt.show()
```



2. Axes Class

Axes class is the most basic and flexible unit for creating sub-plots. A given figure may contain many axes, but a given axes can only be present in one figure. The axes() function creates the axes object.

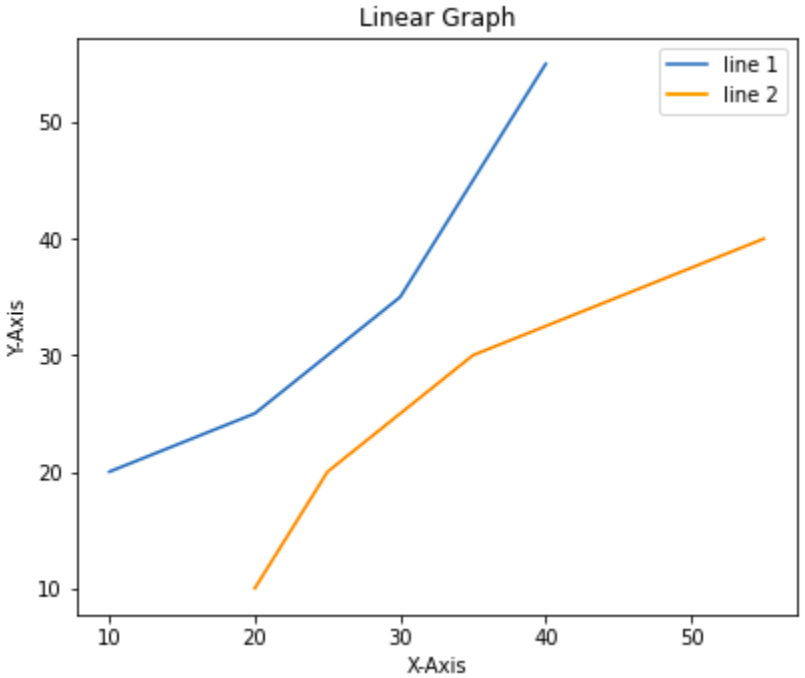
Syntax:

axes([left, bottom, width, height])

Just like pyplot class, axes class also provides methods for adding titles, legends, limits, labels, etc. Let's see a few of them –

- **Adding Title –** ax.set_title()
- **Adding X Label and Y label –** ax.set_xlabel(), ax.set_ylabel()
- **Setting Limits –** ax.set_xlim(), ax.set_ylim()
- **Tick labels –** ax.set_xticklabels(), ax.set_yticklabels()
- **Adding Legends –** ax.legend()

```
# Python program to show pyplot module
import matplotlib.pyplot as plt
from matplotlib.figure import Figure

# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

fig = plt.figure(figsize = (5, 4))

# Adding the axes to the figure
ax = fig.add_axes([1, 1, 1, 1])

# plotting 1st dataset to the figure
ax1 = ax.plot(x, y)

# plotting 2nd dataset to the figure
ax2 = ax.plot(y, x)

# Setting Title
ax.set_title("Linear Graph")

# Setting Label
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")

# Adding Legend
ax.legend(labels = ('line 1', 'line 2'))

plt.show()
```



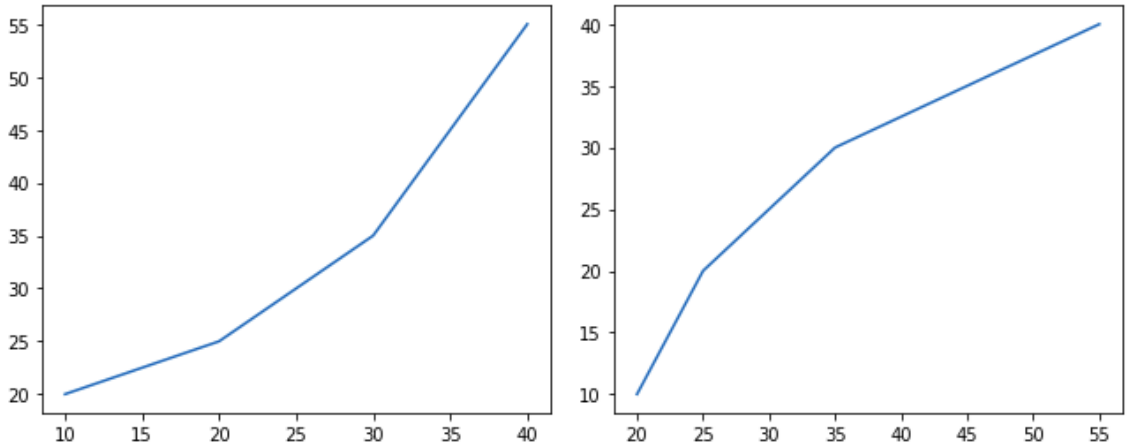
Advanced Plotting: Techniques for Visualizing Subplots

We have learned about the basic components of a graph that can be added so that it can convey more information. One method can be by calling the plot function again and again with a different set of values as shown in the above example. Now let's see how to plot multiple graphs using some functions and also how to plot subplots.

Method 1: Using the add_axes() method

The add_axes() method is used to add axes to the figure. This is a method of figure class

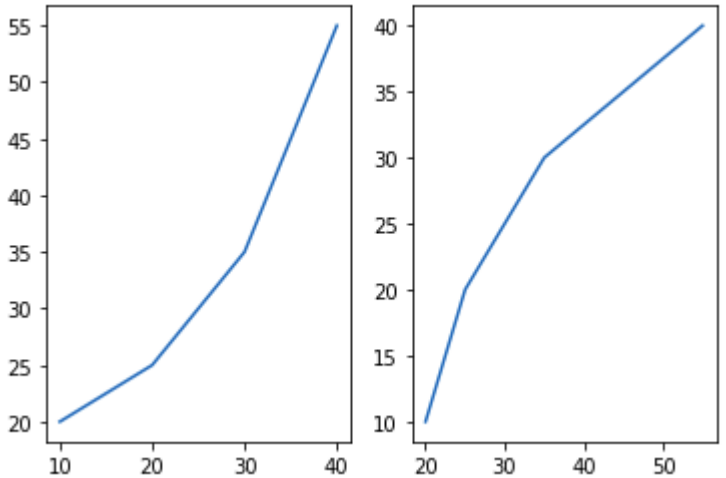
```
# Python program to show pyplot module
import matplotlib.pyplot as plt
from matplotlib.figure import Figure

# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

# Creating a new figure with width = 5 inches
# and height = 4 inches
fig = plt.figure(figsize =(5, 4))

# Creating first axes for the figure
ax1 = fig.add_axes([0.1, 0.1, 0.8, 0.8])

# Creating second axes for the figure
ax2 = fig.add_axes([1, 0.1, 0.8, 0.8])

# Adding the data to be plotted
ax1.plot(x, y)
ax2.plot(y, x)

plt.show()
```



Method 2: Using subplot() method

This method adds another plot at the specified grid position in the current figure.

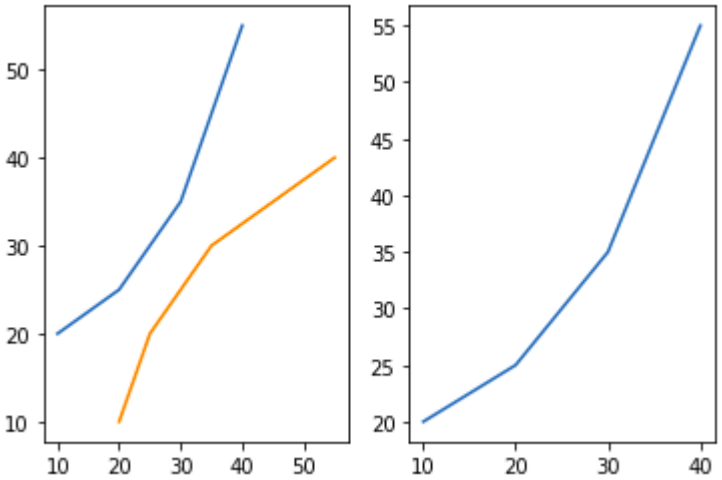
```
import matplotlib.pyplot as plt


# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]


# Creating figure object
plt.figure()
# adding first subplot
plt.subplot(121)
plt.plot(x, y)
# adding second subplot
plt.subplot(122)
plt.plot(y, x)
```



Method 3: Using subplots() method

This function is used to create figures and multiple subplots at the same time.

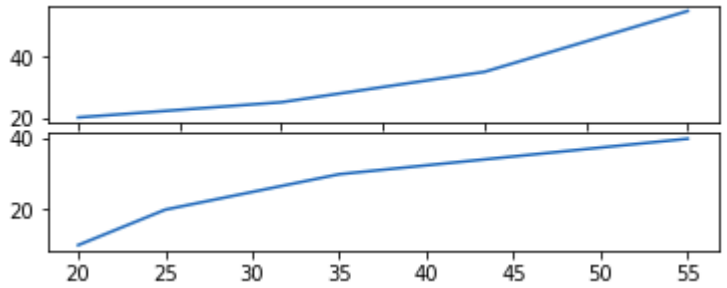
```
import matplotlib.pyplot as plt
# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]
# Creating the figure and subplots
# according the argument passed
fig, axes = plt.subplots(1, 2)
# plotting the data in the
# 1st subplot
axes[0].plot(x, y)
# plotting the data in the 1st
# subplot only
axes[0].plot(y, x)
# plotting the data in the 2nd
# subplot only
axes[1].plot(x, y)
```



Method 4: Using subplot2grid() method

This function creates axes object at a specified location inside a grid and also helps in spanning the axes object across multiple rows or columns. In simpler words, this function is used to create multiple charts within the same figure.

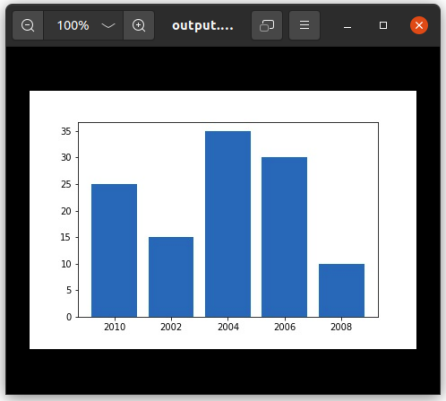
```
import matplotlib.pyplot as plt


# initializing the data
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]
# adding the subplots
axes1 = plt.subplot2grid (
(7, 1), (0, 0), rowspan = 2, colspan = 1)
axes2 = plt.subplot2grid (
(7, 1), (2, 0), rowspan = 2, colspan = 1)
# plotting the data
axes1.plot(x, y)
axes2.plot(y, x)
```

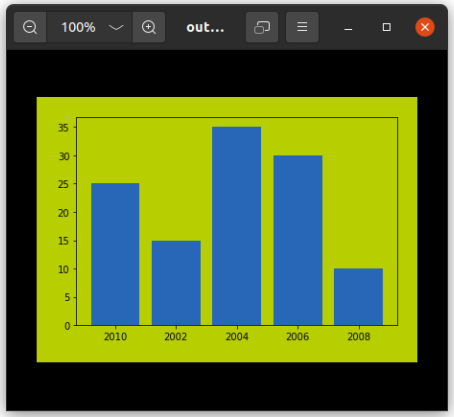


Saving the Matplotlib Plots Using `savefig()`

For saving a plot in a file on storage disk, savefig() method is used. A file can be saved in many formats like .png, .jpg, .pdf, etc.

```
import matplotlib.pyplot as plt


# Creating data
year = ['2010', '2002', '2004', '2006', '2008']
production = [25, 15, 35, 30, 10]


# Plotting barchart
plt.bar(year, production)


# Saving the figure.
plt.savefig("output.jpg")


# Saving figure by changing parameter values
plt.savefig("output1", facecolor='y', bbox_inches="tight",
            pad_inches=0.3, transparent=True)
```





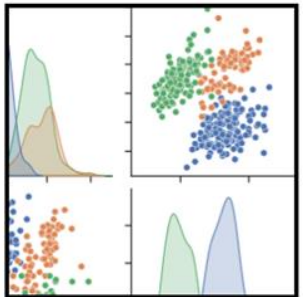
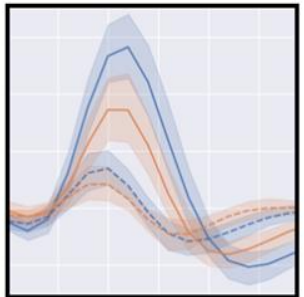
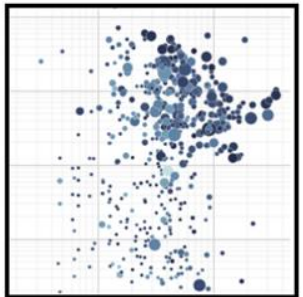
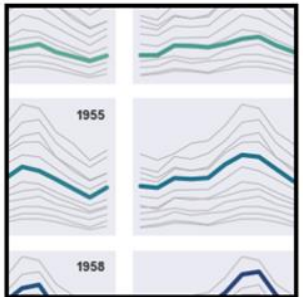
Data Visualization with Seaborn

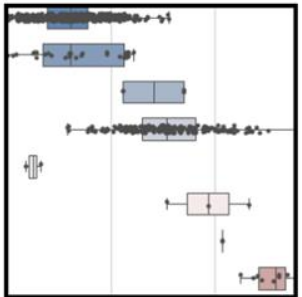
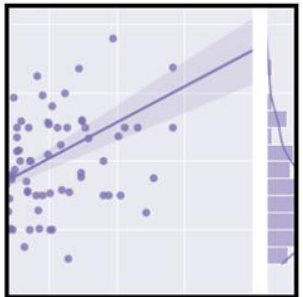
-Statistical data visualization

Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

Data Visualization is the presentation of data in pictorial format.

It is extremely important for Data Analysis, primarily because of the fantastic ecosystem of data-centric Python packages. And it helps to understand the data, however, complex it is, the significance of data by summarizing and presenting a huge amount of data in a simple and easy-to-understand format and helps communicate information clearly and effectively.

Seaborn is a Python data visualization library that simplifies the process of creating complex visualizations.

It is specifically designed for statistical data visualization, making it easier to understand data distributions and relationships between variables.

Seaborn integrates closely with Pandas data structures, allowing seamless data manipulation and visualization.

It is built on the top of matplotlib library and also closely integrated into the data structures from pandas.

Key Features of Seaborn:

- **High-level interface**: Simplifies the creation of complex visualizations.
- **Integration with Pandas**: Works seamlessly with Pandas DataFrames for data manipulation.
- **Built-in themes**: Offers attractive default themes and color palettes.
- **Statistical plots**: Provides various plot types to visualize statistical relationships and distributions.

Installing Seaborn for Data Visualization

pip install seaborn

Creating Basic Plots with Seaborn

Before starting let's have a small intro of bivariate and univariate data:

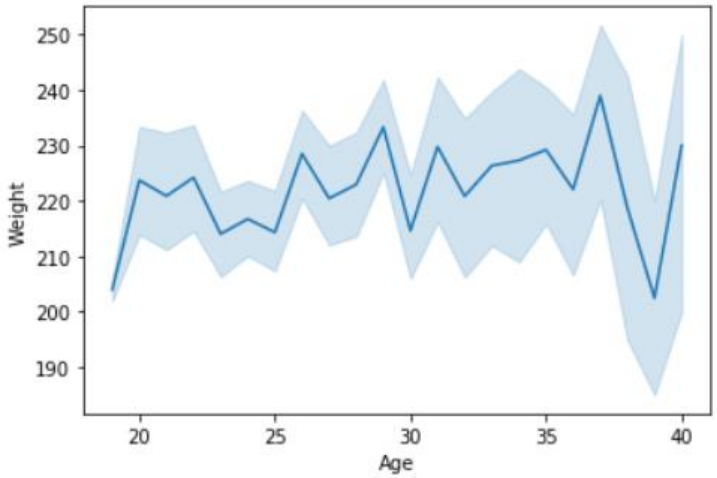
- **Bivariate data:** This type of data involves **two different variables**. The analysis of this type of data deals with causes and relationships and the analysis is done to find out the relationship between the two variables.
- **Univariate data:** This type of data consists of **only one variable**. The analysis of univariate data is thus the simplest form of analysis since the information deals with only one quantity that changes. It does not deal with causes or relationships and the main purpose of the analysis is to describe the data and find patterns that exist within it.

Seaborn offers a variety of plot types to visualize different aspects of data. ***Seaborn helps to visualize the statistical relationships, to understand how variables in a dataset are related to one another and how that relationship is dependent on other variables, we perform statistical analysis.*** This Statistical analysis helps to visualize the trends and identify various patterns in the dataset.

1. Line plot

Lineplot is the most popular plot to draw a relationship between x and y with the possibility of several semantic groupings. It is often used to track changes over intervals.

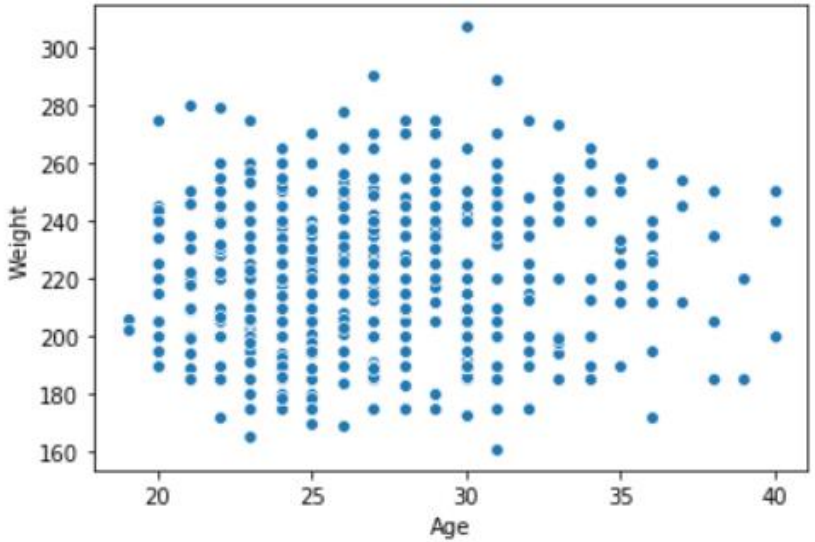
```
import pandas as pd

import seaborn as sns


# initialise data of lists

data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ],

        'Age':[ 30 , 21 , 29 , 28 ]}

df = pd.DataFrame( data )


# plotting lineplot

sns.lineplot( data['Age'], data['Weight'])
```



2. Scatter Plot

Scatter plots are used to visualize the relationship between two numerical variables. They help identify correlations or patterns. It can draw a two-dimensional graph.

```
import pandas as pd

import seaborn as sns


# initialise data of lists

data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ],

        'Age':[ 30 , 21 , 29 , 28 ]}

df = pd.DataFrame( data )


seaborn.scatterplot(data['Age'],data['Weight'])
```



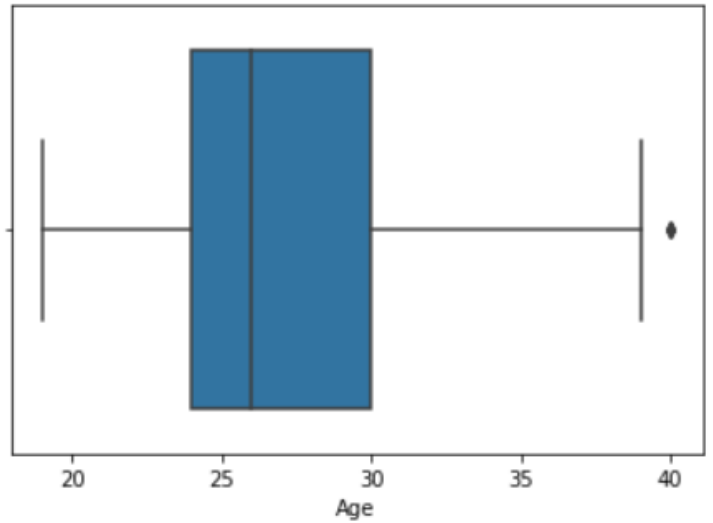
3. Box plot

A box plot (or box-and-whisker plot) s is the visual representation of the depicting groups of numerical data through their quartiles against continuous/categorical data.

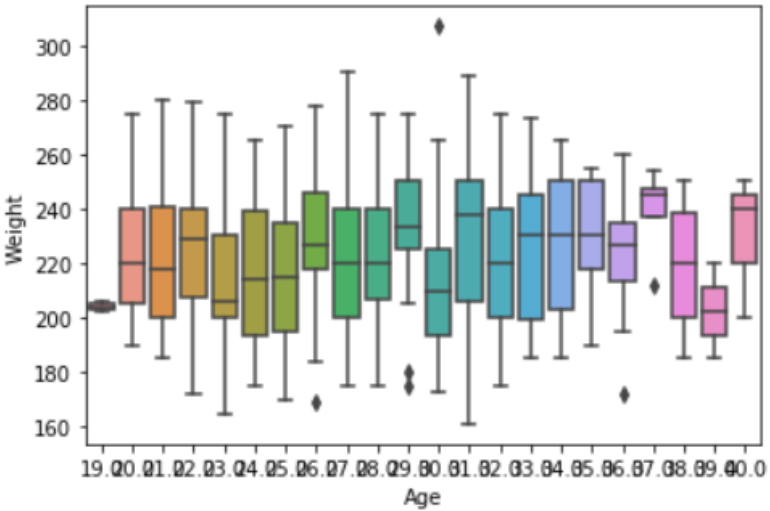
A box plot consists of 5 things.

- Minimum
- First Quartile or 25%
- Median (Second Quartile) or 50%
- Third Quartile or 75%
- Maximum

```
import pandas as pd
import seaborn as sns

# initialise data of lists
data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ],
        'Age':[ 30 , 21 , 29 , 28 ]}
df = pd.DataFrame( data )
sns.boxplot( data['Age'] )
```



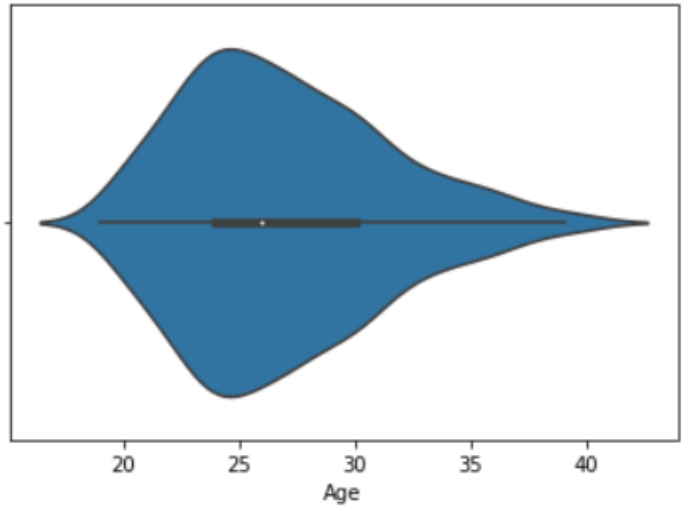
```
# import module
import seaborn as sns
import pandas

# read csv and plotting
data = pandas.read_csv( "nba.csv" )
sns.boxplot( data['Age'], data['Weight'])
```



4. Violin Plot

A violin plot is similar to a boxplot. It shows several quantitative data across one or more categorical variables such that those distributions can be compared.

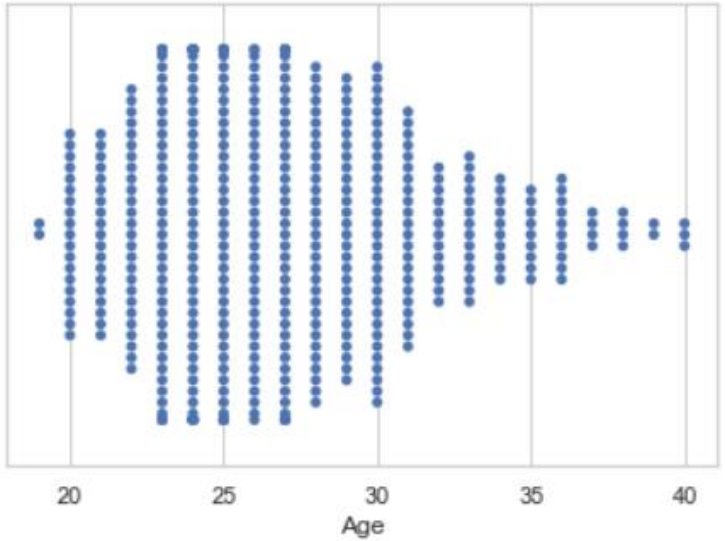
```
import pandas as pd
import seaborn as sns


# initialise data of lists
data = {'Name':[ 'Mohe' , 'Karnal' , 'Yrik' , 'jack' ],
        'Age':[ 30 , 21 , 29 , 28 ]}
df = pd.DataFrame( data )
sns.violinplot(data['Age'])
```



5. Swarm plot

A swarm plot is similar to a strip plot, We can draw a swarm plot with non-overlapping points against categorical data.

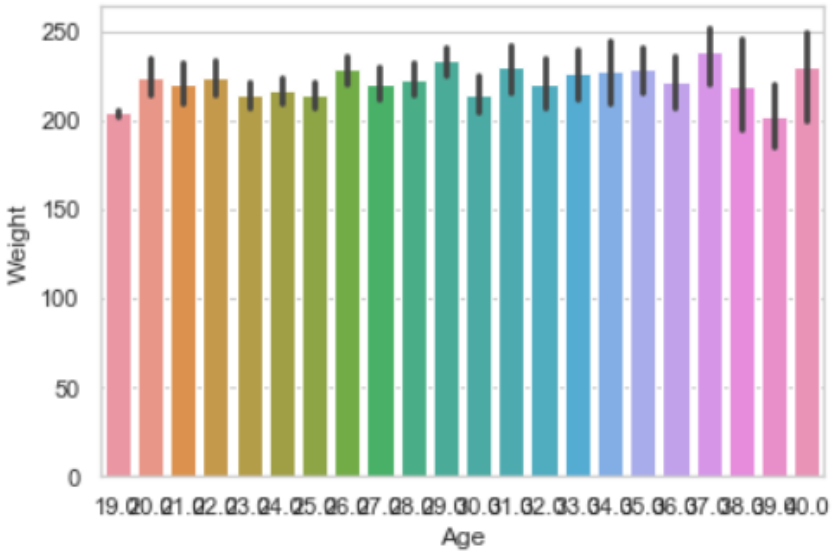
```
# import module
import seaborn


seaborn.set(style = 'whitegrid')


# read csv and plot
data = pandas.read_csv( "nba.csv" )
seaborn.swarmplot(x = data["Age"])
```



6. Bar plot

Barplot represents an estimate of central tendency for a numeric variable with the height of each rectangle and provides some indication of the uncertainty around that estimate using error bars.

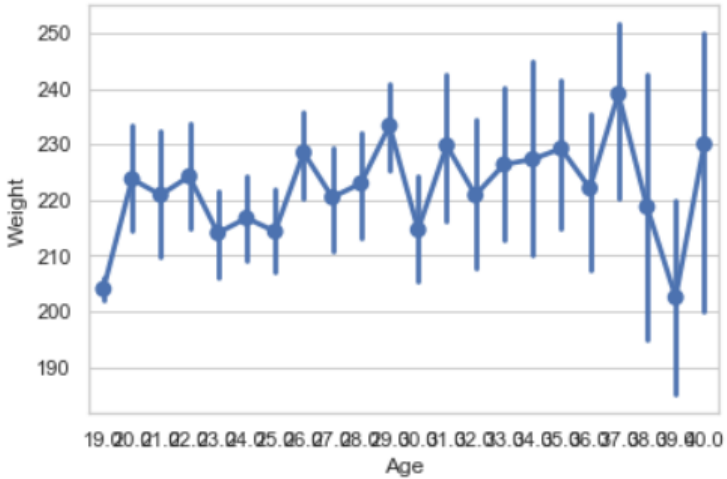
```
# import module
import seaborn


seaborn.set(style = 'whitegrid')


# read csv and plot
data = pandas.read_csv("nba.csv")
seaborn.barplot(x ="Age", y ="Weight", data = data)
```



7. Point plot

Point plot used to show point estimates and confidence intervals using scatter plot glyphs. A point plot represents an estimate of central tendency for a numeric variable by the position of scatter plot points and provides some indication of the uncertainty around that estimate using error bars.

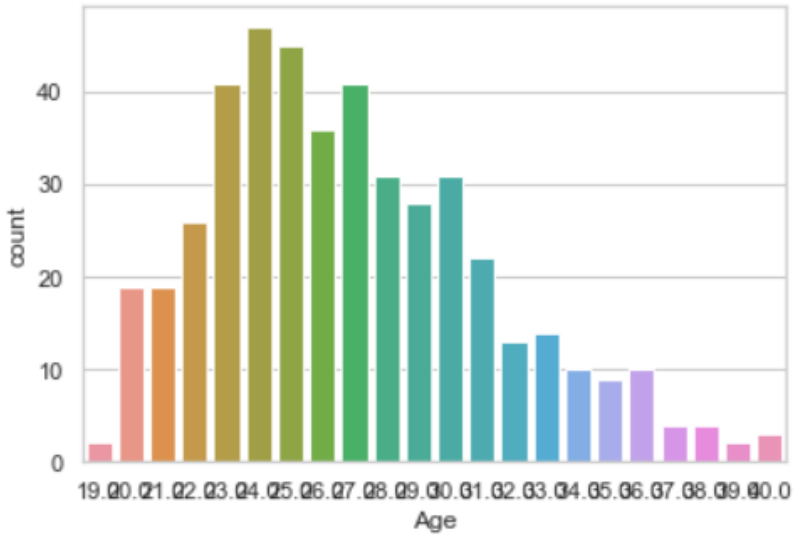
```
# import module
import seaborn


seaborn.set(style = 'whitegrid')


# read csv and plot
data = pandas.read_csv("nba.csv")
seaborn.pointplot(x = "Age", y = "Weight", data = data)
```



8. Count plot

Count plot used to Show the counts of observations in each categorical bin using bars.

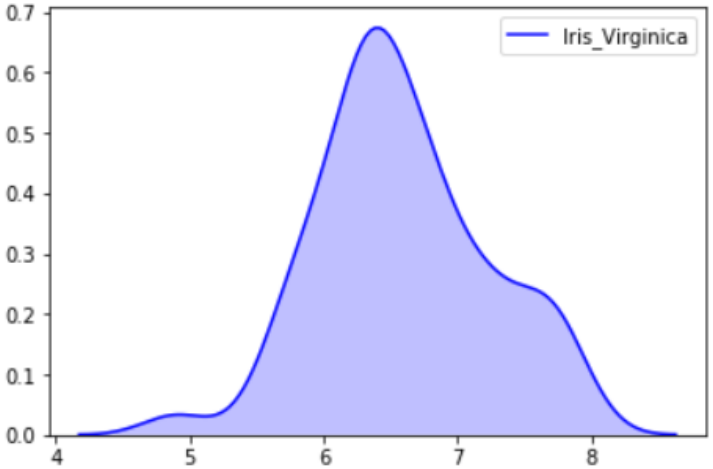
```
# import module
import seaborn


seaborn.set(style = 'whitegrid')


# read csv and plot
data = pandas.read_csv("nba.csv")
seaborn.countplot(data["Age"])
```



9. KDE Plot

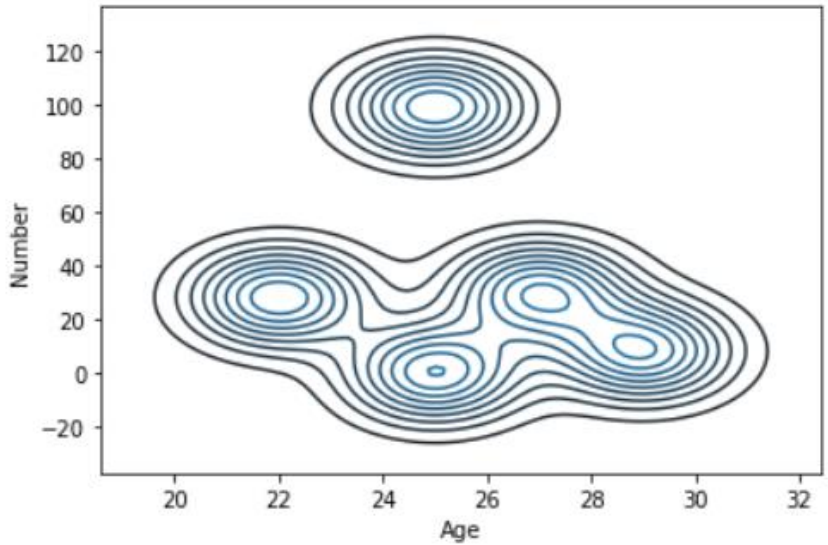
KDE Plot described as **Kernel Density Estimate** is used for visualizing the Probability Density of a continuous variable. It depicts the probability density at different values in a continuous variable. We can also plot a single graph for multiple samples which helps in more efficient data visualization.

```
# importing the required libraries

from sklearn import datasets

import pandas as pd

import seaborn as sns

# Setting up the Data Frame

iris = datasets.load_iris()

iris_df = pd.DataFrame(iris.data, columns=['Sepal_Length',

                    'Sepal_Width', 'Patal_Length', 'Petal_Width'])


iris_df['Target'] = iris.target

iris_df['Target'].replace([0], 'Iris_Setosa', inplace=True)

iris_df['Target'].replace([1], 'Iris_Vercicolor', inplace=True)

iris_df['Target'].replace([2], 'Iris_Virginica', inplace=True)

  # Plotting the KDE Plot

sns.kdeplot(iris_df.loc[(iris_df['Target'] =='Iris_Virginica'),

          'Sepal_Length'], color = 'b', shade = True, Label ='Iris_Virginica')
```



Example 2: KDE plot for Age and Number Feature

```
# import module
import seaborn as sns
import pandas

# read top 5 column
data = pandas.read_csv("nba.csv").head()

sns.kdeplot( data['Age'], data['Number'])
```

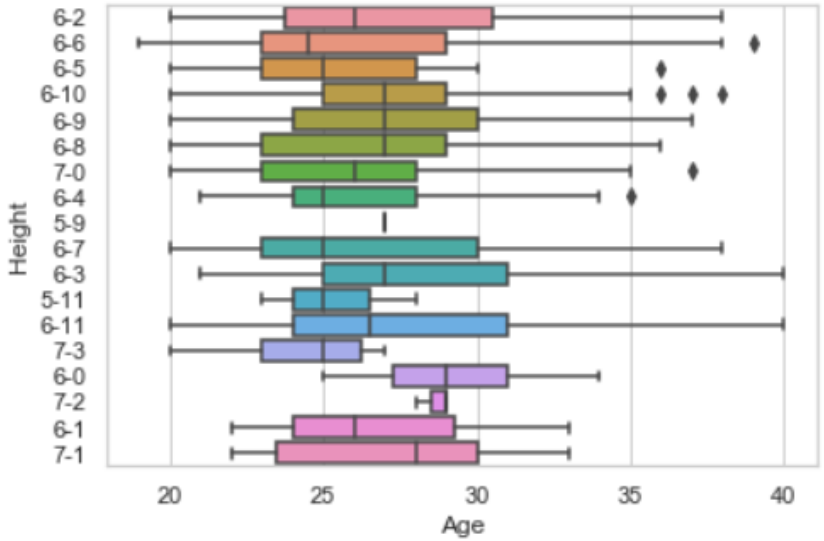


Bivariate and Univariate data Using Seaborn

Let's see an example of **Bivariate data :**

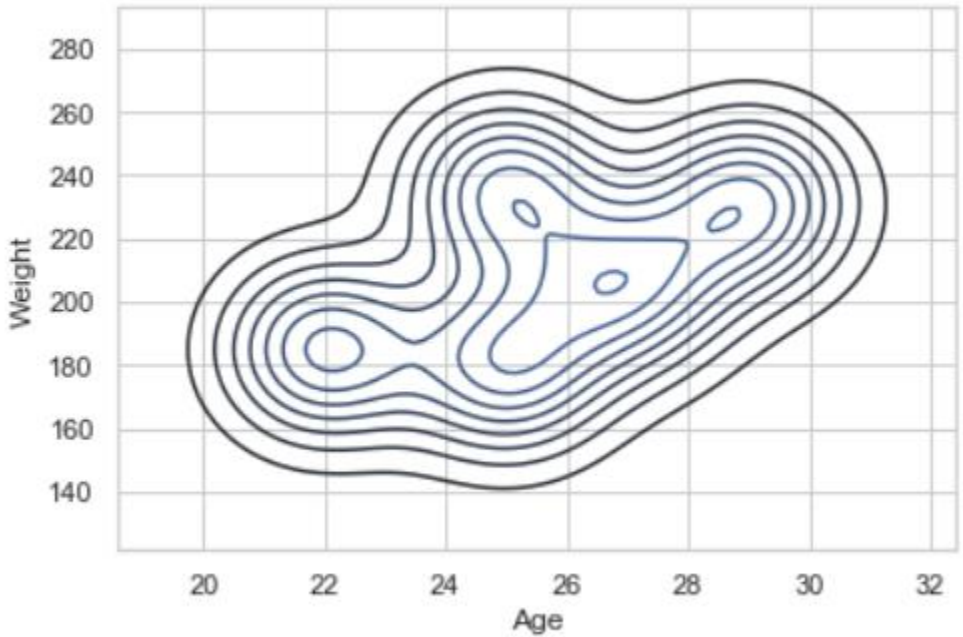
Example 1: Using the box plot.

```
# import module

import seaborn as sns

import pandas


# read csv and plotting

data = pandas.read_csv( "nba.csv" )

sns.boxplot( data['Age'], data['Height'])
```



Example 2: Using KDE plot

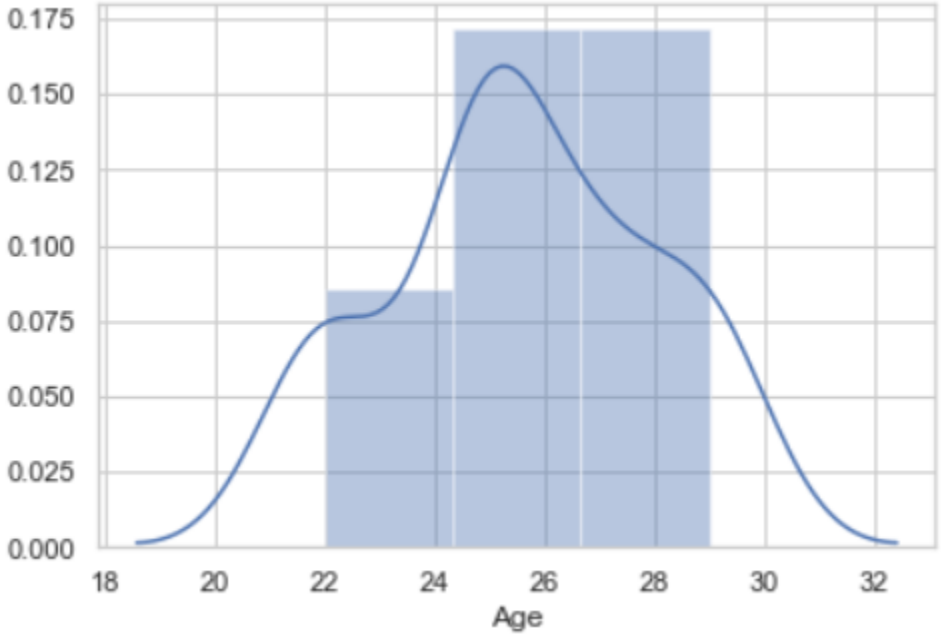
```
# import module
import seaborn as sns
import pandas

# read top 5 column
data = pandas.read_csv("nba.csv").head()

sns.kdeplot( data['Age'], data['Weight'])
```



example of **univariate data distribution**

Example 1: Using the dist plot

```
# import module

import seaborn as sns

import pandas


# read top 5 column

data = pandas.read_csv("nba.csv").head()


sns.distplot( data['Age'])
```

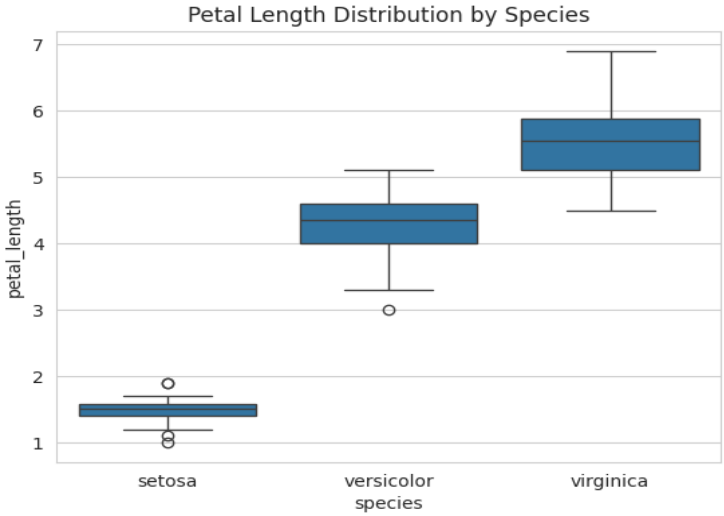


Customizing Seaborn Plots with Python

Seaborn plots can be customized extensively to improve their readability and aesthetics.

1. Changing Plot Style and Theme

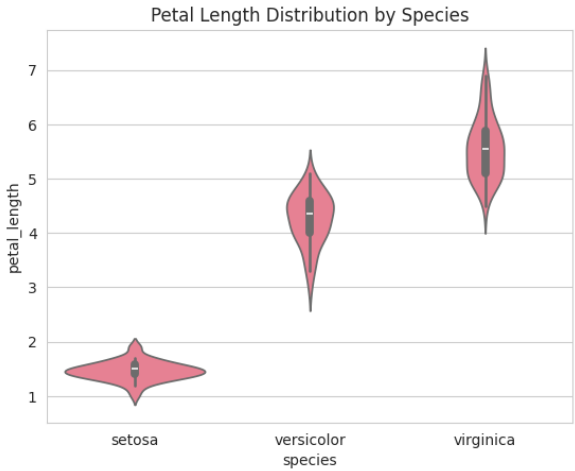
Seaborn offers several built-in themes that can be used to change the overall look of the plots. These themes include `darkgrid`, `whitegrid`, `dark`, `white`, and `ticks`.




```
import seaborn as sns
import matplotlib.pyplot as plt


# Set the style of the plots
sns.set_style("whitegrid")


# Example plot with the selected style
sns.boxplot(x='species', y='petal_length', data=sns.load_dataset('iris'))
plt.title('Petal Length Distribution by Species')
plt.show()
```

2. Customizing Color Palettes

Seaborn allows you to use different color palettes to enhance the visual appeal of your plots. You can use predefined palettes or create custom ones.

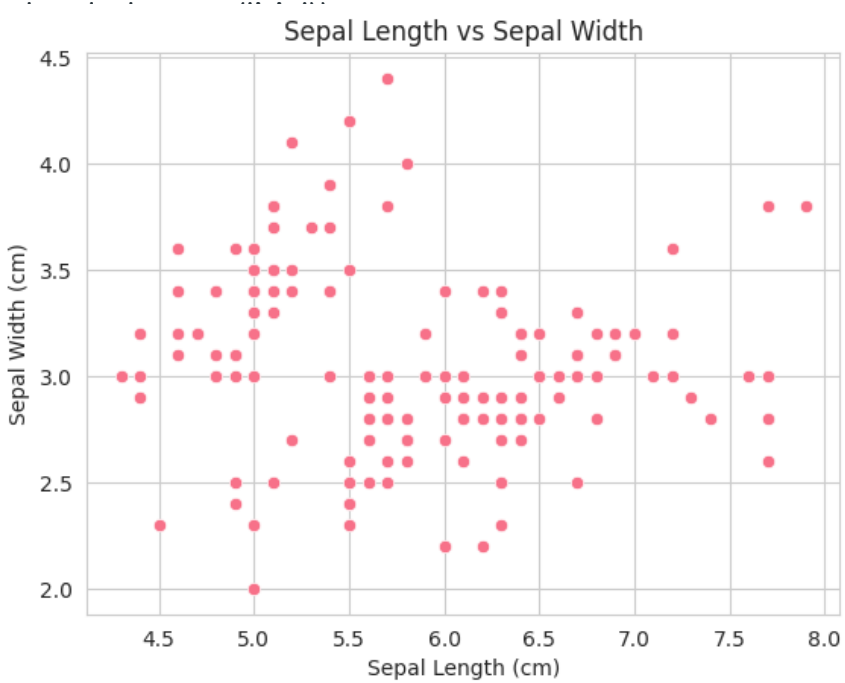



```
# Set a custom color palette
custom_palette = sns.color_palette("husl", 8)


# Apply the custom palette
sns.set_palette(custom_palette)


# Example plot with the custom palette
sns.violinplot(x='species', y='petal_length', data=sns.load_dataset('iris'))
plt.title('Petal Length Distribution by Species')
plt.show()
```

3. Adding Titles and Axis Labels

Adding descriptive titles and labels to your plots can make them more informative.

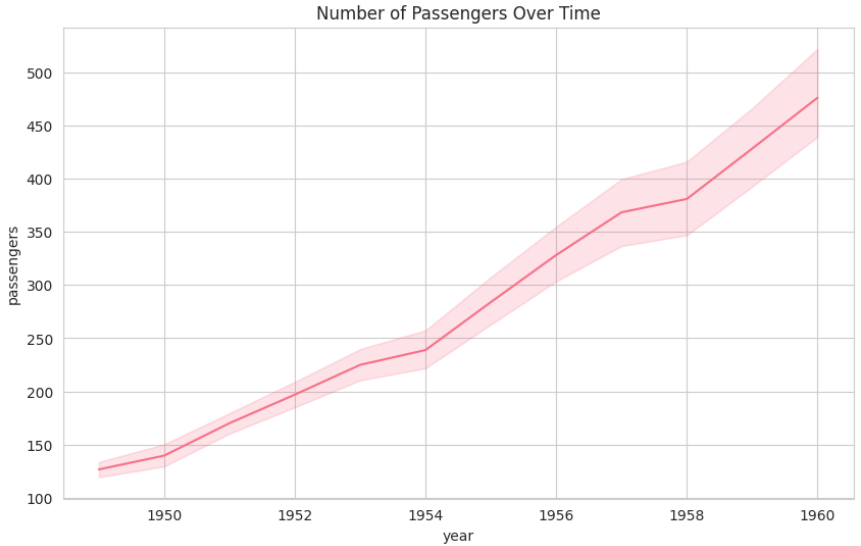
```
# Adding title and labels
sns.scatterplot(x='sepal_length', y='sepal_width', data=sr
plt.title('Sepal Length vs Sepal Width')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```



4. Adjusting Figure Size and Aspect Ratio

You can adjust the size of the figure to make it fit better in your presentations or reports.

```
# Adjust figure size
plt.figure(figsize=(10, 6))


# Example plot with adjusted figure size
sns.lineplot(x='year', y='passengers', data=sns.load_dataset
plt.title('Number of Passengers Over Time')
plt.show()
```



Covariance and Correlation

Covariance and correlation are about the relationship between the variables. Covariance defines the *directional association* between the variables. Covariance values range from *-inf* to *+inf* where a positive value denotes that both the variables move in the same direction and a negative value denotes that both the variables move in opposite directions.

Correlation is a standardized statistical measure that expresses the extent to which two variables are linearly related (meaning how much they change together at a constant rate). The *strength and directional association* of the relationship between two variables are defined by correlation and it ranges from -1 to +1. Similar to covariance, a positive value denotes that both variables move in the same direction whereas a negative value tells us that they move in opposite directions.

Advanced Data Visualization-Interactive Plots with Plotly.

Tableau is a fantastic tool for creating interactive visualizations, it is a bit of a hassle to switch between platforms: training models and creating segments in the Python environment and then viewing the results in another tool.
Moreover, for professional use it is chargeable.

Plotly Express vs Plotly Go

The Plotly Python library is an interactive, open-source plotting library that covers a wide range of graph types and data visualization use cases. It has a wrapper called Plotly Express that is a higher-level interface to Plotly.

Plotly Express is quick and easy to use as a starting point for creating the most common figures with simple syntax, but it lacks functionality and flexibility when it comes to more advanced chart types or customizations.

Unlike Plotly Express, Plotly Go (Graph Objects) is a lower-level graph package that generally requires more code, but is much more customizable and flexible. In this tutorial, we will use Plotly Go to create an interactive scatter bubble graph similar to the one created in Tableau. You can also save the coding as a template to create similar charts in other use cases.